AN INDEPENDENT COMPUTATIONAL VERIFICATION OF LOWER BOUNDS ON THE NUMBER OF PRIME FACTORS OF QUASIPERFECT NUMBERS

ANSEL MARSH

Abstract. A quasiperfect number is a positive integer n with $\sigma(n) = 2n + 1$; none is known. By exhaustive computer search we show that every quasiperfect number has at least eight distinct prime factors (a repeat with corrected rounding bounds is in progress; Section 11.4). This independently confirms a bound announced by Toyohara, Tao and Yao (2026). Under hypotheses on the small primes we obtain $\omega(n) \geq 18$ when $\gcd(n, 15) = 1$, improving the $\omega(n) \geq 15$ of Prasad and Sunitha (2017), together with $\omega(n) \geq 10$ when $3 \nmid n$ and $\omega(n) \geq 8$ when $5 \nmid n$. The search is an exact branch-and-bound over odd squares with $\sigma(n) = 2n + c$. Because solutions exist for $c > 1$, every prune can be tested against real solutions. Nodes with two primes left are handled by a compiled search in double-double arithmetic, with every survivor settled exactly. This brings the case $\omega = 7$ from an estimated 21 838 or more core-hours down to about 61; the hardest part lies next to the Fermat prime 65537. We also reproduce $\omega(n) \geq 7$ (Hagis and Cohen, 1982) independently. The results assume Cattaneo's theorem that n is an odd square, and nobody else has verified them.

1. Introduction

Let $\sigma(n)$ denote the sum of the positive divisors of n and $h(n) = \sigma(n)/n$ its *abundancy*. A number is *perfect* if $\sigma(n) = 2n$, and *quasiperfect* if $\sigma(n) = 2n + 1$, that is, if it exceeds perfection by the least possible amount. No quasiperfect number is known, and none has been ruled out.

Cattaneo [2] showed that a quasiperfect number must be an odd perfect square. Hagis and Cohen [3] showed that it must exceed 10^{35} and have at least seven distinct prime factors; both bounds stood for over four decades. In 2026 Alekseyev [1] raised the size bound to 10^{45}, and Toyohara, Tao and Yao [7] announced that $\omega(n) \geq 8$, noting that an earlier computation by Zemann [9] towards the same bound had a gap in its coverage of exponents.

Stronger bounds are known under hypotheses on the small primes. Abbott et al. showed $\omega(n) \geq 10$ when $\gcd(n, 15) = 1$, and Prasad and Sunitha [4] raised that to $\omega(n) \geq 15$, also giving an alternative proof of the general bound when $\gcd(n, 15) = 5$. These are conditional, and the condition matters for what follows: our computation finds that the expensive part of the search is precisely where 3 and 5 *do* divide n (Section 7), which is exactly the case those bounds do not reach. The two literatures are complementary — what is cheap to search is what is already known by other means, and vice versa.

Results of this kind are established by exhaustive computer search, and they are only as reliable as the proof that the search covers every case. That makes independent reproduction valuable, particularly with different code and different methods. This paper describes such an independent search and its verification.

Contributions.

Date: Draft of 19 October 2026.

This paper, the code behind it and the computations it reports were produced by an AI system. See the statement at the end.

(1) A complete search of $\omega = 7$, giving $\omega(n) \geq 8$ for every quasiperfect number (Theorem 19). This independently confirms the bound announced in [7], by different methods and code (Section 10).
(2) Conditional bounds we cannot find in the literature: $\omega(n) \geq 8$ for $5 \nmid n$, $\omega(n) \geq 10$ for $3 \nmid n$, and $\omega(n) \geq 18$ for $\gcd(n, 15) = 1$, improving 15 [4] (Section 6).
(3) A consequence with no hypothesis: any quasiperfect number with exactly seven distinct prime factors is divisible by 15 (Proposition 15), found first and now subsumed by Theorem 19.
(4) An independent reproduction of $\omega(n) \geq 7$ [3], by a method built from the problem rather than from that paper, with coverage audited separately from the run and the result obtained three times by different decompositions (Section 5).
(5) The search itself: every prune a stated lemma with a proof (Section 3), exact arithmetic throughout, algebraic closure of the leaves on which enumeration does not terminate, and proofs that the tree is exhaustive (Proposition 2) and finite (Proposition 3).
(6) A method of testing it. Since the search solves $\sigma(n) = 2n + c$ for any c, and solutions exist for $c > 1$, every prune can be checked against real solutions rather than only against itself (Section 4).
(7) Independent derivations of results we then found in the literature: the Legendre–Cattaneo filter (Lemma 11), worth a factor of 130 here with its corollary, and the three lemmas underlying the 2026 computation (Section 9), which make $\omega = 7$ affordable on one desktop.
(8) That the cost of a subtree is predictable before searching it (Section 8.4), which makes the tractable part of an intractable search accessible and localises the rest.
(9) A full record of the errors, dead ends and misjudgements of the project (Section 12), including two cases where a test was constructed so as to be incapable of failing.

2. Setup and notation

Throughout, $c \geq 1$ is fixed and we seek odd squares

$$n = \prod_{i=1}^{k} p_i^{2a_i}, \qquad 3 \leq p_1 < \cdots < p_k, \quad a_i \geq 1,$$

with

$$\sigma(n) = 2n + c, \quad \text{equivalently} \quad h(n) = 2 + \frac{c}{n}. \tag{1}$$

For $c = 1$ the restriction to odd squares loses nothing. Half of that is immediate, and we record it because the same argument is reused in Lemma 10.

Lemma 1. *If $\sigma(n)$ is odd then $n = 2^a m^2$ with m odd. In particular a quasiperfect number has this form.*

Proof. σ is multiplicative and $\sigma(p^e) = 1 + p + \cdots + p^e$ is a sum of $e + 1$ terms. For odd p every term is odd, so $\sigma(p^e)$ is odd exactly when e is even; for $p = 2$, $\sigma(2^e) = 2^{e+1} - 1$ is always odd. Hence $\sigma(n)$ is odd precisely when every odd prime of n occurs to an even power. For a quasiperfect number $\sigma(n) = 2n + 1$ is odd. □

That $a = 0$, that is, that a quasiperfect number is odd, is the substantive part of Cattaneo's theorem [2] and we use it as cited rather than reproducing it; the parity argument above does not settle it, since $\sigma(m^2) \equiv 3 \pmod 4$ is not by itself impossible. The search is therefore run over odd squares on the strength of [2], and this is the one input to our results that we have not verified ourselves. A reader who wished to avoid that dependence could run the same search over $n = 2^a m^2$ with $a \geq 1$ as an additional family; we have not done so.

We use that h is multiplicative and that for an odd prime p,

$$h(p^{2a}) = \frac{p^{2a+1}-1}{p^{2a}(p-1)}$$

increases strictly with a, to the limit $p/(p-1)$. Also $h(p^{2a}) \geq 1 + 1/p + 1/p^2$.

2.1. **Search states.** A *state* consists of a list of distinct odd primes $p_1 < \cdots < p_j$, each either *fixed* at exponent $2a_i$ or *open* with exponent at least $2A_i$, together with the number $r = k - j$ of primes still to be chosen, all of which must exceed p_j. For a state we define

$$\mathrm{lo} = \prod_i h(p_i^{2a_i}), \qquad \mathrm{hi} = \prod_{\text{fixed}} h(p_i^{2a_i}) \prod_{\text{open}} \frac{p_i}{p_i - 1}, \qquad n_{\min} = \prod_i p_i^{2a_i},$$

where for open components a_i means A_i. If n is any solution compatible with the state, and m is its part over $p_1, \ldots, p_j$, then $\mathrm{lo} \leq h(m)$, $n_{\min} \leq m$, and $h(m) < \mathrm{hi}$ whenever an open component exists (otherwise $h(m) = \mathrm{lo} = \mathrm{hi}$).

2.2. **The tree.** Say that a solution n (an odd square with $\sigma(n) = 2n+c$ and $\omega(n) = k$) is *compatible* with a state if n's first j primes in increasing order are exactly $p_1 < \cdots < p_j$, each fixed component's exponent in n equals its $2a_i$, and each open component's exponent in n is at least its $2A_i$. The root is the empty state with $r = k$, and every solution is compatible with it.

A state is expanded in one of three ways:

- *add*: choose the next prime $q > p_j$, entered as open with $A = 1$;
- *refine*: replace an open component $(p, \geq 2A)$ by the two children $(p, 2A)$ fixed and $(p, \geq 2A + 2)$ open;
- *close*: resolve the state exactly by Lemma 9 or Lemma 8.

Proposition 2 (Completeness)**.** *Let n be a solution compatible with a state S. If S is expanded by* refine*, then n is compatible with exactly one of the two children. If S is expanded by* add*, then n is compatible with the child for $q =$ the smallest prime of n exceeding p_j, and that q is among those the search enumerates. If S is closed, n is among the numbers the closing lemma returns. Consequently a completed search that returns no solution proves that none exists with that ω.*

Proof. Refine: n's exponent of p is at least $2A$; it either equals $2A$, matching the fixed child and not the open one, or is at least $2A + 2$, matching the open child and not the fixed one. Where the exponent equals $2A$, the fixed child survives the tests of Lemmas 5 and 11, since both are necessary conditions on an exact component of a solution.

Add: since $\omega(n) = k$ and j primes are fixed with $r = k - j \geq 1$ remaining, n has a smallest prime q beyond p_j, and n is compatible with that child by definition. It remains to check that no prune excludes it. The lower cut on q is Lemma 7(2) and the upper cut Lemma 7(3), both of which every solution satisfies; the test applied to each individual q is Lemma 7(1); and q survives Lemma 5. In the branch where $\mathrm{lo} \geq 2$ the enumerated range is that of Lemma 7(4). Each of these is a necessary condition on a solution, so none of them can discard n.

Close: Lemmas 9 and 8 are stated as equivalences or as necessary conditions with the resulting finite candidate set searched exhaustively, so n is returned.

Finally, a state is pruned only by Lemma 6 or Lemma 7, whose hypotheses are contradicted by any compatible solution. Hence if a solution existed, it would be compatible with the root and, by induction along the branch chosen above, with a leaf that reports it. □

Proposition 3 (Termination)**.** *The search visits finitely many states.*

Proof. Only *refine* can be applied unboundedly often at fixed j, since *add* strictly decreases r and *close* ends a branch. So it suffices to show that no branch refines forever. Suppose it did. Then

some component stays open with $A \to \infty$, and along that branch lo increases monotonically to the limit

$$L = \prod_{i \notin O} h(p_i^{2a_i}) \prod_{i \in O} \frac{p_i}{p_i - 1},$$

where O is the set of components refined infinitely often. Also hi $\geq L$ throughout, with hi constant once no component outside O changes.

By Lemma 10, $L \neq 2$. If $L > 2$, then lo exceeds 2 after finitely many refinements, and also exceeds $2 + c/n_{\min}$, since $n_{\min}$ grows without bound; the branch is then pruned by Lemma 6 (if $r = 0$) or Lemma 7(4) (if $r \geq 1$, noting $c/(n_{\min} \operatorname{lo}) \to 0$ so no prime is admissible). If $L < 2$ then hi < 2 after finitely many refinements, so the state is no longer eligible for the hi ≥ 2 refinement rule; with $r = 0$ and one component open it is closed by Lemma 8, and with $r \geq 1$ the remaining refinement rules are governed by the ratio test of Section 3, which fails once lo is close enough to L. Either way the branch ends, a contradiction. □

Remark 4. Proposition 3 is not a formality. The first version of the refinement rule described in Section 12 (Issue 8) tested only hi, which for an open component does not change as its exponent bound rises; that rule satisfied every condition of Proposition 2 but refined without end. Completeness and termination must be checked separately.

3. The lemmas

Lemma 5 (Coprimality)**.** *If n satisfies* (1) *and $p^{2a} \parallel n$, then every prime dividing both n and $\sigma(p^{2a})$ divides c. In particular, for $c = 1$, no prime factor of n divides $\sigma(p^{2a})$.*

Proof. $\gcd(n, \sigma(n)) = \gcd(n, 2n + c) = \gcd(n, c)$, and $\sigma(p^{2a})$ divides $\sigma(n)$ by multiplicativity. □

Lemma 6 (Complete states)**.** *Let $r = 0$. If* hi ≤ 2 *and some component is open, or if* lo $> 2 + c/n_{\min}$*, then no solution is compatible with the state.*

Proof. A solution has $h(n) = 2 + c/n$, which is > 2 and $\leq 2 + c/n_{\min}$. With an open component $h(n) <$ hi, and always $h(n) \geq$ lo. □

Lemma 7 (Next prime)**.** *Let $r \geq 1$ and let q be the smallest prime of n not yet chosen. Then*

1. lo $\cdot(1 + 1/q + 1/q^2) \leq 2 + c/(n_{\min} q^2)$*;*
2. $q >$ lo $/(2 -$ lo $+ c/n_{\min})$ *when* lo < 2*;*
3. hi $\cdot \prod_t t/(t-1) > 2$*, the product running over q and the $r - 1$ consecutive primes following it;*
4. *if* lo ≥ 2 *then $q < c/(n_{\min}$* lo*$)$. In particular, for $c = 1$ no state with $r \geq 1$ and* lo ≥ 2 *has a solution.*

Proof. Write $n = mu$ with m the part over the chosen primes. Then $q^2 \mid u$, so $h(u) \geq h(q^{2b}) \geq 1 + 1/q + 1/q^2$ and $n \geq mq^2 \geq n_{\min} q^2$. Hence $\operatorname{lo}(1 + 1/q + 1/q^2) \leq h(m)h(u) = h(n) = 2 + c/n \leq 2 + c/(n_{\min} q^2)$, which is (1). Dropping $1/q^2$ on the left and weakening the right to $2 + c/n_{\min}$ gives lo $+$ lo $/q < 2 + c/n_{\min}$, which rearranges to (2). For (3), $h(u) < \prod_{t \mid u} t/(t-1)$, and replacing the r primes of u by the r smallest primes that are at least q can only increase this product; since $h(n) > 2$ the bound follows. For (4), lo$(1 + 1/q) \geq 2 +$ lo $/q$, so (1) gives lo $/q < c/(n_{\min} q^2)$. □

Part (3) gives an upper limit for q whenever hi < 2, since the product decreases to 1 as q grows. When hi $\geq 2 >$ lo some component is open, and the search refines it before adding a prime.

Lemma 8 (One open exponent)**.** *Let $n = n' p^{2a}$ with $p \nmid n'$ satisfy* (1)*. Then*

$$p^{2a} \mid \sigma(n') + c(p - 1).$$

In particular, when every other component is fixed, the exponent of p is bounded by $\frac{1}{2} v_p\big(\sigma(n') + c(p-1)\big)$.

Proof. Modulo p^{2a} we have $\sigma(n) = 2n + c \equiv c$, and $(p-1)\sigma(p^{2a}) = p^{2a+1} - 1 \equiv -1$. Multiplying $\sigma(n) = \sigma(n')\sigma(p^{2a})$ by $p-1$ gives $c(p-1) \equiv -\sigma(n') \pmod{p^{2a}}$. The right-hand side $\sigma(n') + c(p-1)$ is positive, so its p-adic valuation is finite. □

Lemma 9 (Last prime)**.** *Let m be odd with $S = \sigma(m) < 2m$, put $D = 2m - S$, and let $q \nmid m$ be a prime. Then $n = mq^{2a}$ satisfies* (1) *if and only if*

$$P_a(q) := q^{2a}(2m - Dq) - cq - (S - c) = 0. \tag{2}$$

If $S \geq c$, every such q satisfies $q \leq (2m-1)/D$ and $q^{2a} \leq cq + S - c$.

Proof. Multiply $S \cdot (q^{2a+1} - 1)/(q-1) = 2mq^{2a} + c$ by $q - 1$ and rearrange. If $S \geq c$ the right side $cq + S - c$ of $q^{2a}(2m - Dq) = cq + S - c$ is positive, so $2m - Dq \geq 1$. □

For each a, the integer roots of P_a are found exactly. Since

$$P_a''(q) = 2a\, q^{2a-2}\big((2a-1)2m - (2a+1)Dq\big)$$

changes sign once, at $q^* = (2a-1)2m/((2a+1)D)$, the derivative P_a' increases up to q^* and decreases after it. So P_a has at most three monotone stretches on $(0, \infty)$. Their endpoints are located by bisection on P_a', and each stretch is bisected for a root. Only finitely many a need checking: once $(p_j + 1)^{2a} > c(2m-1)/D + S$, no admissible q satisfies the second bound of the lemma.

Lemma 10 (Parity)**.** *For any odd primes p_i, integers $a_i \geq 1$ and any subset O of indices,*

$$\prod_{i \notin O} h(p_i^{2a_i}) \prod_{i \in O} \frac{p_i}{p_i - 1} \neq 2.$$

Proof. Written as a single fraction, the numerator $\prod_{i \notin O} \sigma(p_i^{2a_i}) \prod_{i \in O} p_i$ is odd, since $\sigma(p^{2a})$ is a sum of $2a + 1$ odd terms. The denominator is $\prod_{i \notin O} p_i^{2a_i} \prod_{i \in O}(p_i - 1)$. If $O \neq \emptyset$ it is even, and odd/even $\neq 2$. If $O = \emptyset$ the product equals $\sigma(m)/m$ with m odd, and $\sigma(m) = 2m$ is impossible because $\sigma(m)$ is odd. □

Lemma 11 (Quadratic residue)**.** *Let $n = m^2$ be an odd square with $\sigma(n) = 2n + c$, and let p^{2a} be the exact power of p dividing n. Then every prime r dividing $\sigma(p^{2a})$ with $r \nmid 2c$ satisfies*

$$\left(\frac{-2c}{r}\right) = 1.$$

For $c = 1$ this says $r \equiv 1$ or $3 \pmod 8$.

Proof. By multiplicativity $\sigma(p^{2a}) \mid \sigma(n)$, so $r \mid 2m^2 + c$, giving $2m^2 \equiv -c \pmod r$. Multiplying by 2,

$$(2m)^2 \equiv -2c \pmod r.$$

If $r \nmid 2c$ then $-2c$ is a non-zero square modulo r, so its Legendre symbol is 1. For $c = 1$, $\left(\frac{-2}{r}\right) = \left(\frac{-1}{r}\right)\left(\frac{2}{r}\right)$, which is $+1$ exactly for $r \equiv 1, 3 \pmod 8$ and -1 for $r \equiv 5, 7 \pmod 8$. □

Lemma 11 is the strongest prune we have, because it applies to a single component in isolation: a pair $(p, 2a)$ can be rejected outright, with no reference to the rest of the state. Taking $c = 1$ and $a = 1$:

$$\sigma(3^2) = 13, \quad \sigma(5^2) = 31, \quad \sigma(257^2) = 61 \cdot 1087, \quad \sigma(65537^2) = 37 \cdot 116085511,$$

and $13 \equiv 5$, $31 \equiv 7$, $61 \equiv 5$, $37 \equiv 5 \pmod 8$. So none of 3^2, 5^2, 257^2, 65537^2 can be the exact power of its prime in a quasiperfect number, while $\sigma(17^2) = 307 \equiv 3$ is permitted. Section 7 draws the consequence for the hardest states.

Lemma 11 is not new, and we make no claim of priority. It was, however, derived here independently: stated, proved, implemented, tested for soundness against solutions with $c > 1$, and measured at a factor of 130 (Section 5), before we found it in the literature as the *Legendre–Cattaneo*

filter — in exactly the form used here, including its application to each component $\sigma(p^{2a})$, and formally verified in Lean 4 by the UALBF project [5]. It is *not* the "quadratic-residue sieve" of [7], as we previously guessed it was: theirs tests whether a discriminant is a quadratic residue to several small moduli, sieving candidate primes before an exact perfect-square check, which is a condition on a different object altogether (Section 9).

That sequence is worth recording for two reasons beyond candour. First it is checkable rather than merely asserted: the development history, the timestamped source variants of Section 13.4 and the git history together fix the order of events. Second, the convergence is evidence. An independent derivation reaching a known classical result, in the same unusual per-component form, corroborates both the derivation and the implementation built on it — the same sort of check as the two unrelated decompositions of Section 5.2 agreeing on the exact state count, and rather stronger, since the agreement here is with the literature rather than with ourselves. What it does not do is make the lemma ours, and Section 12 records why we should have found it sooner.

Remark 12. In implementation the test is deliberately one-sided. Deciding it exactly needs the full factorisation of $\sigma(p^{2a})$, which is infeasible for large p and a, so we factor only as far as is cheap and reject only when a violating prime is actually exhibited. Failing to factor therefore costs pruning power and never costs soundness. The cheap Jacobi test $\left(\frac{-2c}{\sigma}\right) = -1$ is a useful first filter but is not equivalent: two violating primes can cancel, as in $\sigma(257^2) = 61 \cdot 1087$ where both factors are bad yet the Jacobi symbol of the product is $+1$.

3.1. **A free corollary: the mod-8 exponent test.** Lemma 11 requires factoring $\sigma(p^{2a})$, which is why the implementation caps the effort and so sometimes fails to reject a component it should. A weaker form of it costs nothing at all.

Corollary 13. *For $c = 1$, if p^{2a} is the exact power of p dividing n then*

$$\sigma(p^{2a}) \equiv 1 + a(p+1) \equiv 1 \text{ or } 3 \pmod 8.$$

Proof. The residues $\{1, 3\}$ are closed under multiplication modulo 8, since $3 \cdot 3 = 9 \equiv 1$. By Lemma 11 every prime factor of $\sigma(p^{2a})$ lies in that set, so the product does too. For the explicit form, $p^2 \equiv 1 \pmod 8$ for odd p, so of the $2a+1$ terms of $\sigma(p^{2a})$ the $a+1$ even powers are each $\equiv 1$ and the a odd powers are each $\equiv p$, giving $\sigma(p^{2a}) \equiv (a+1) + ap = 1 + a(p+1)$. □

This is a test on a and p mod 8 alone: no factoring, no large arithmetic. The admissible exponents are $a \equiv 0, 1 \pmod 4$ for $p \equiv 1 \pmod 8$; a even for $p \equiv 3 \pmod 8$; $a \equiv 0, 3 \pmod 4$ for $p \equiv 5 \pmod 8$; and no restriction for $p \equiv 7 \pmod 8$, where $p + 1 \equiv 0$ and the congruence is vacuous. The identity was checked over 2 256 pairs (p, a).

Three of these are the exponent conditions of Cattaneo's 1951 theorem [2], which we derived here before finding them there and which we had in any case read only from a damaged extraction of that paper — it disagrees with the derivation for $p \equiv 5 \pmod 8$, and we follow the derivation. Cattaneo's fourth condition, $a \equiv 1$ for $p \equiv 7 \pmod 8$, does not follow from this argument even working modulo 16, and we do not use it.

That last point was checked rather than assumed, since modulo 16 looked like the obvious place to find the missing condition. It is not there, and the reason is structural: the residues $\{1, 3, 9, 11\}$ modulo 16 — exactly those congruent to 1 or 3 modulo 8 — are closed under multiplication, forming a cyclic group generated by 3. The argument above therefore reproduces itself one power higher and yields nothing new per component, which we confirmed numerically over more than two thousand pairs (p, a): not one is rejected modulo 16 that survives modulo 8. For $p \equiv 7 \pmod 8$ in particular, $\sigma(p^{2a})$ is 1 or 9 modulo 16 for every a, both admissible.

Working modulo 16 does yield one thing, but a global one. An odd square is 1 or 9 modulo 16, so for $c = 1$

$$\sigma(n) = 2n + 1 \equiv 3 \pmod{16}$$

exactly, for every candidate. Writing each component's level $\lambda(p, 2a)$ for the exponent with $3^\lambda \equiv \sigma(p^{2a}) \pmod{16}$, the levels of a complete factorisation must sum to 1 modulo 4. This is a genuine necessary condition and it is one we have not seen stated, but it is a single equation over $\mathbb{Z}/4$ on the *complete* component set, so it constrains only the final component: a four-fold cut on leaves. As Section 9.6 records, leaves are where this search spends almost none of its time, so we verified the condition and did not implement it.

Being a consequence of Lemma 11 the corollary adds no reach in principle, but it adds a great deal in practice, because it fires where the factoring gives up: at $\omega = 6$ it cuts the state count from 20 211 100 to 15 353 292 and the time from twenty-one minutes to eight. It is guarded to $c = 1$: for other c the admissible residues are those with $\left(\frac{-2c}{r}\right) = 1$, which need not be closed under multiplication, so the step from each factor to the product fails.

Lemma 10 is what makes refinement terminate. As the open exponents of a state grow, lo increases to the limit on the left-hand side, which is either below 2 or above it. In the first case $\mathrm{hi} < 2$, and the search may add a prime (or, when $r = 0$, prune the state). In the second, lo eventually exceeds 2, or $2 + c/n_{\min}$, and the state is pruned by Lemma 7(4) or Lemma 6.

3.2. **Order of operations.** At a state with $r \geq 1$: prune by Lemma 7(4) and (3); if $\mathrm{hi} \geq 2$, refine; if $r \geq 2$, some component is open and $r\,\mathrm{hi}\,/(2 - \mathrm{hi}) > 5000$ (so the next prime would range over a very long interval), refine; if $r = 1$ and no component is open, close by Lemma 9; if $r = 1$ and the admissible range for q (Lemma 7(2),(3)) is wider than a fixed threshold, refine; otherwise add each admissible q, subject to Lemmas 5 and 7(1). Whenever a component is fixed at exponent $2a$, it must pass Lemmas 5 and 11. At a state with $r = 0$: if nothing is open, test (1) directly; otherwise prune by Lemma 6, and close by Lemma 8 if exactly one component is open, or refine the open component with the least exponent if several are.

The threshold rule for $r = 1$ affects efficiency only. Its motivation: as the open intervals tighten, the admissible range for the last prime shrinks to a width close to 1, so refining first avoids enumerating a wide range of last primes with exponents still undetermined. Applying the same rule at $r \geq 2$ was tried and made the search roughly ten times slower.

3.3. **A worked example.** The case $c = 1$, $k = 3$ shows the bounds doing their work, and is small enough to check by hand. Write $f(p) = p/(p - 1)$.

At the root, $r = 3$ and Lemma 7(3) requires $f(p_1)f(p_2)f(p_3) > 2$ over the three smallest admissible primes. For $p_1 = 3$ this is $f(3)f(5)f(7) = \frac{3}{2} \cdot \frac{5}{4} \cdot \frac{7}{6} = \frac{105}{48} = 2.1875 > 2$, so $p_1 = 3$ survives; for $p_1 = 5$ it is $f(5)f(7)f(11) = \frac{385}{240} \approx 1.604 < 2$, so every branch with $p_1 \geq 5$ is dead at once. Hence $3 \mid n$.

Enter 3 as open, so $\mathrm{lo} = h(3^2) = \frac{13}{9}$, $\mathrm{hi} = f(3) = \frac{3}{2}$, $n_{\min} = 9$, and $r = 2$. Since $\mathrm{hi} < 2$ no refinement is forced. For the next prime q, Lemma 7(3) now requires $\mathrm{hi}\, f(q)f(q') > 2$ with q' the prime after q: for $q = 5$, $\frac{3}{2} \cdot \frac{5}{4} \cdot \frac{7}{6} = 2.1875 > 2$, but for $q = 7$, $\frac{3}{2} \cdot \frac{7}{6} \cdot \frac{11}{10} = 1.925 < 2$. So $q = 5$ is forced and $15 \mid n$.

Now $\mathrm{lo} = \frac{13}{9} \cdot \frac{31}{25} = \frac{403}{225}$, $\mathrm{hi} = \frac{3}{2} \cdot \frac{5}{4} = \frac{15}{8}$, $n_{\min} = 225$, $r = 1$. The last prime is confined from below by Lemma 7(2),

$$q > \frac{\mathrm{lo}}{2 - \mathrm{lo} + 1/n_{\min}} = \frac{403/225}{(450 - 403 + 1)/225} = \frac{403}{48} \approx 8.40,$$

and from above by Lemma 7(3), which gives $f(q) > 2/\,\mathrm{hi} = \frac{16}{15}$, that is $q < 16$. Only $q = 11$ and $q = 13$ remain, and both pass the sharper test of Lemma 7(1) and the coprimality test of Lemma 5 (neither divides $\sigma(3^2) = 13$ or $\sigma(5^2) = 31$). Each of the two surviving states is then refined until Lemma 10 forces hi below 2 or lo above $2 + 1/n_{\min}$, and both die. The whole of $\omega = 3$ costs eight states.

The pattern generalises: the upper bound of Lemma 7(3) makes the smallest primes almost forced, while the admissible window for the last prime is what grows explosively when the fixed part's abundancy approaches 2. That is the mechanism behind the Fermat-prime subtree of Issue 7.

4. Verification

The implementation is about 300 lines of Python using exact rational and integer arithmetic throughout. Primality of candidate primes is decided by Miller–Rabin with the first twenty prime bases. This can only err by calling a composite prime, which adds a spurious branch and cannot remove a genuine one.

4.1. **Brute force.** For every odd $k \le 30001$ we computed $c = \sigma(k^2) - 2k^2$. For each pair (c, ω) with $0 < c \le 2 \times 10^5$ found this way (thirteen pairs, with $\omega \in \{3, 4\}$), the search was run with that c and ω. In every case the solutions it returned that were at most 30001^2 coincided exactly with the brute-force list, and no state was left unresolved.

4.2. **Planted solutions.** Brute force cannot reach large exponents or large last primes, which is exactly where the closing lemmas operate. We therefore built random odd squares $n = \prod p_i^{2a_i}$ with two to five primes, exponents $2a_i$ up to 24 and a largest prime up to 2000. For each n with $c = \sigma(n) - 2n > 0$, the closing routines were asked to recover the missing piece of n:

- *Last prime*: all components but the largest prime fixed, whenever that remainder satisfies $\sigma(m) < 2m$ (Lemma 9).
- *One exponent*: all primes known and one exponent, chosen at random, open from 2 (Lemma 8).

Across four random seeds, 8 000 instances of each type were recovered with no failures. The planted values of c are far larger than 1, which exercises parts of the last-prime root finding that never arise for $c = 1$.

4.3. **Internal consistency.** The parallel driver, which cuts the tree at a fixed depth into independent subtrees, reproduces the serial node count exactly for $\omega = 5$ (193 559 nodes). The tool that splits a single subtree further (Issue 7) was checked the same way, against an already completed subtree whose node count it reproduced exactly (200 601).

4.4. **Soundness of the quadratic-residue prune.** Lemma 11 is the only prune that rejects a component without reference to the rest of the state, so an error in it would silently remove solutions rather than produce a visible unresolved state. It was therefore tested three ways.

First, directly: the lemma predicts that 13, 31, 61 and 37 never divide $2m^2 + 1$, while 11 and 307 (both $\equiv 3 \bmod 8$) may. Over $m \le 2 \times 10^5$ the former divide it not once — indeed 0 is not among the residues of $2m^2 + 1$ modulo 13 — and over $m \le 2 \times 10^4$ the latter divide it 3 636 and 130 times respectively.

Second, against real solutions. The lemma generalises to arbitrary c, which matters because solutions exist for $c > 1$ and can be found by brute force. Of the 37 genuine solutions of $\sigma(n) = 2n + c$ with $c \le 10^6$ and $m \le 8000$, and all 131 of their components, not one is rejected. Any rejection would have been a counterexample.

Third, inside the search. With the prune enabled the brute-force comparison of Section 4 still recovers every known solution for all thirteen (c, ω) pairs, with no mismatches and nothing unresolved, and the planted-leaf suites still recover all their instances. Node counts drop while results do not change, which is the signature wanted of a sound prune.

4.5. **Testing the faulty versions.** The defects described in Section 12 are claims about particular versions of the program, so they are tested as such. Each recovered faulty variant (Section 13.4) is run against the defect attributed to it, alongside its replacement on the same input; the results appear with the issues themselves. This is not redundant with the suites above, which test only the current version: it is what stops the narrative of Section 12 from being unfalsifiable. In one case it showed our first test was measuring nothing (Issue 8).

5. Results

ω	nodes	solutions	unresolved states
1	1	0	0
2	1	0	0
3	8	0	0
4	542	0	0
5	193 559	0	0
6	1 998 951 360	0	0
7	partial only; see Section 8		

Table 1. Search for $\sigma(n) = 2n + 1$ with n an odd square, by number of distinct prime factors, *without* Lemma 11. A run establishes its row only if the unresolved column is zero.

ω	states without Lemma 11	with it	reduction
3	8	7	1.1×
4	542	208	2.6×
5	193 559	15 176	12.8×
6	1 998 951 360	15 353 292	130×

Table 2. Effect of Lemma 11 together with its free corollary, the mod-8 exponent test of Section 3.1. Every row still reports no solution and no unresolved state. The reduction grows with ω, as one would expect of a test applied independently to each component.

Both runs of each row agree: no solutions, nothing unresolved. So $\omega = 6$ has now been settled three times over by this implementation — once as 358 parallel subtrees, once with its hardest subtree decomposed a second way (Section 5.2), and once in a single process with the prunes enabled. The last took eight minutes on one core against 17.4 hours on fourteen, a reduction of about 288 in total processor time. Notably its exact leaf closures fell from 335 to zero: with the prune in place, no branch survives long enough to need one.

Theorem 14. *No quasiperfect number has six or fewer distinct prime factors.*

This is not new: it is one of the two bounds of Hagis and Cohen [3], and its value here lies only in the independence of the method and the code. It is worth being exact about how much of that paper is reproduced, which is less than the phrase "the 1982 bound" suggests. Theorem 14 corresponds to their prime-factor bound alone. Their size bound $n > 10^{35}$ rests on different techniques and is untouched by anything here; since raised to 10^{45} by Alekseyev [1], it is the natural next target and is discussed in Section 14. We have also not examined whatever subsidiary results their paper contains beyond these two, so no claim is made about them. And the search assumes throughout

that a quasiperfect number is an odd square, which is Cattaneo's theorem [2], cited and not reproved (Lemma 1 and the remark following it). Our result is therefore conditional on that one prior result holding.

5.1. **The shape of the $\omega = 6$ computation.** The tree was cut at four primes into 358 independent subtrees. The work is extraordinarily lopsided: 208 of them finish in under ten seconds, the slowest of the ordinary ones takes 2.6 hours, and a single subtree — the one beginning $3, 5, 17, 257$ of Issue 7 — accounts for 1.405×10^9 states, just over 70% of the whole computation. Together the run took about 17.4 hours of wall-clock time and, counting each task once, some 38 core-hours. Only 335 exact leaf closures were needed in the whole of $\omega = 6$: almost everything dies to the bounds of Lemma 7 long before a leaf is reached.

Coverage was audited separately from the run, by re-deriving the list of subtrees from the splitter and checking that every one has a record, that no record belongs to a subtree outside the list, and that no record reports a solution or an unresolved state. That audit passes, which is what turns "nothing was found" into Theorem 14 via Proposition 2.

5.2. **An unplanned replication.** The concurrency of Issue 9 produced a stronger check than we designed. The Fermat subtree was searched twice to completion by different routes: once as a single process walking it whole, and once as 12 195 separate processes, each handling one candidate fifth prime, with the enumeration of that prime lifted into the splitter. The two runs report exactly the same number of search states, 1 404 698 494. Since the decomposition is textually unrelated to the traversal — different process structure, different order, different entry points — an agreement to the last digit is good evidence that the tree is well defined and that the implementation walks it deterministically.

The two routes are not equally efficient, and the difference measures Issue 8 directly. The split consumed 218 103 core-seconds against the single process's 62 605: a factor 3.5 more work for the same states, because each of the 12 195 pieces re-refines the exponents of $3, 5, 17, 257$ from scratch. In exchange it finished in 16 779 seconds of wall-clock time against 62 605, a speedup of 3.7 on thirteen cores (29% efficiency). So the split bought a useful reduction in wall-clock time at a large cost in total work, and the refinement rule of Issue 8 addresses exactly the waste it introduced.

6. Conditional bounds

The results of this section may be new. They are also the cheapest computations in the paper, for a reason that is worth stating plainly: barring small primes is exactly the hypothesis that makes this method work well, and the literature appears not to have searched those cases beyond the elementary count.

6.1. **Why the restricted cases are easy.** Everything expensive in Sections 7 and 8 came from configurations in which $\prod p/(p-1)$ sits within 10^{-5} of 2, which requires the small primes. Remove them and the arithmetic changes character. Reading Lemma 7(3) as a bound on ω by restricting the available primes:

The published record for the restricted cases, taken from the introduction of [4] rather than from any summary of it, is: $\omega \geq 5$ (Abbott, Aull, Brown and Suryanarayana, Theorem 2), improved to 6 by Kishore and to 7 by Cohen and Hagis; and conditionally $\omega \geq 10$ if $\gcd(N, 15) = 1$ (the same authors, Theorem 5), refined to 15 by [4], who also give an alternative proof of the unconditional bound when $\gcd(N, 15) = 5$. We can find no bound stated for $3 \nmid n$ with 5 permitted, so for that hypothesis the unconditional bound is what stands.

The last row is the opening. Unconditionally, search has bought four or five beyond the elementary bound; in the coprime-to-15 case the published bound *is* the elementary bound, so no search appears to have been applied there at all.

primes barred	$\omega \geq$	best published
none	3	7 [3], 8 claimed [7]
5	4	none found; the unconditional bound applies
3	7	none found; the unconditional bound applies
3 and 5	15	15 [4], refining 10 (Abbott et al., 1973)

TABLE 3. Primes needed for $\prod p/(p-1) > 2$ once small primes are barred, against the best bounds we can find in the literature. Note that $\omega \geq 15$ for $\gcd(n, 15) = 1$ is exactly the elementary count.

6.2. **Results.** Running the search of this paper with those primes barred gives the following. Each line means the case was eliminated with no solution found and no state left unresolved.

hypothesis	ω eliminated	states	new bound	previous
$5 \nmid n$	4	24		
	5	703		
	6	41 267		
	7	19 899 890	$\omega \geq 8$	7
$3 \nmid n$	7	15		
	8	1 871		
	9	540 250	$\omega \geq 10$	7
$\gcd(n, 15) = 1$	15	2 159		
	16	788 607		
	17	1.51×10^{10}*	$\omega \geq 18$	15 [4]

TABLE 4. Conditional bounds. Values of ω below those listed are excluded by the elementary count of the previous table, in one state each. Cost grows by roughly a factor of 300 per step, so one or two further steps in each case is the practical limit. * 12 916 647 713 states of the exact search plus 2 199 616 513 of the compiled search that closed its one unresolved node (Section 6.3).

These improve the bound for $\gcd(n, 15) = 1$ from 15 to 18, and give 10 for $3 \nmid n$ and 8 for $5 \nmid n$, where only the unconditional bound of 7 previously applied. The last of those is the expensive one, at twenty million states against a few hundred thousand for the others, because permitting 3 readmits the configurations that drive the abundancy towards 2.

6.3. **The last step for** $\gcd(n, 15) = 1$**: closing one node.** The search for $\omega = 17$ with 3 and 5 barred was split into 507 subtrees and run between 2 and 11 October 2026. It took 12 916 647 713 states and found no solution, the last four subtrees each taking between 38 and 152 hours on one core. It did not finish cleanly. One state in the last subtree was left unresolved: fifteen components, nine of them with open exponents, and two primes $q < q'$ still to choose. Enumeration of q stopped at the limit of 10^7 candidates, at $q = 571\,287\,683$, while the admissible range runs to 1 137 878 224, leaving 27 577 412 primes.

Why the remainder is dear. Write $\eta = 2 - \mathrm{hi}$ for the node; here $\eta = 3.515 \times 10^{-9}$. Below $q = 2/\eta \approx 5.69 \times 10^8$ each q forces some open exponents to stay small, and the first ten million q cost about 450 states each. Above it every open exponent may grow without bound for every q, and the exact search spends about 8 000 states and two seconds per q re-refining the same nine exponents: roughly 15 000 core-hours for the remainder. The run had stopped almost exactly where its cost changes character.

Issue: a dead end. Since neighbouring q give almost the same tree, we first carried q as an interval of primes, with bounds valid for every prime in it, and split the interval only when its own spread dominated (`qblock.py`). It agreed with the exact search on the test suites and was slower in practice: the surviving exponent boxes are numerous at every scale, so the shared upper levels saved little and the late splits repeated the work. The cost lies in the number of exponent patterns per q, not in the arithmetic of each.
The fix: the same search, compiled, in logarithms. An open component p with exponent $2a$ contributes $\log\frac{p}{p-1} - d_p(a)$ to the log-abundancy, where $d_p(a) = -\log(1-p^{-(2a+1)})$, so the equation becomes

$$\log f(q) + \log f(q') \;=\; K + \sum_p d_p(a_p) + \log\Big(1+\frac{1}{2N}\Big), \qquad K = \log\frac{2}{\mathrm{hi}},$$

where $\log f(x) = \log\big(\sigma(x^{2b})/x^{2b}\big)$ lies in $[\log(1+1/x+1/x^2),\ -\log(1-1/x))$ for every $b \ge 1$. For each q the program (`qfast.py`, compiled with Numba) keeps a box over the open exponents, each either fixed or open from a floor. It converts the box into the range of q' it permits, and splits the open component with the widest contribution until that range is short. It then tries each prime q' and splits further until the box is narrower than the target the pair leaves. The residue test on fixing an exponent is kept, precomputed per (p, a) with the exact search's own function. The other prunes are dropped, which can only enlarge the tree.
Issue: precision. The terms are about 1.8×10^{-9}, and a first version in double precision needed a slack of 2×10^{-23} on every comparison. It left about 0.56 survivors per q for the exact code at 75 ms each, some 90 core-hours in all. Moving the sums to double-double arithmetic brought the rounding error to about 10^{-41}; K and every $d_p(a)$ are summed from their series as exact rationals and then split into high and low parts. Checked against exact rationals, the computed bounds on $\log f(x)$ are within 1.3×10^{-41}. The slack is 10^{-35}, so a comparison can only keep too much, never discard a solution. Survivors fell to 22 in the whole range.
Result. All 27 577 412 primes q in $(571\,287\,683,\ 1\,137\,878\,224]$ took 2 199 616 513 states in about fifteen minutes on one core. The 22 survivors were handed, as boxes, to the original exact search, which settled them in 1 418 states with no solution and nothing unresolved. With the node closed, $\omega = 17$ is eliminated when $\gcd(n, 15) = 1$, so $\omega(n) \ge 18$.
Validation, and its limit. The compiled search was checked by planting. For 300 random choices of q, of q' up to $50q$, and of a fixed exponent vector, K was set so that the planted point satisfies the equation exactly; every time, a returned box contained it. That tests the splitting and the bounds. It does not test the computation of K for the real node, which rests on the rational series above. As a second, independent check, the original exact code was run on 4 000 primes sampled at random from the same range: 13 172 539 states, no solution and nothing unresolved (`spot_qrange.py`). The compiled search is new code, checked only by planting and sampling, and an independent re-run of this node is the check we would most like to see.
Issue: two slips while running it. Our first estimate for the remainder was "hours" by the discriminant criterion. That was wrong: the criterion needs fixed exponents, and nine were open. And the first full launch spent eighty minutes doing nothing. The end of the q-range had been written as $1/(1-s)$ instead of $s/(s-1)$, with $s = \sqrt{2/\mathrm{hi}}$; the result is negative, and a loop meant to adjust the end by a few units counted downwards indefinitely. The block-search version had the formula right, and it was retyped rather than reused.

6.4. **A consequence for the unconditional problem.** The two one-prime cases combine into a statement with no hypothesis attached.

Proposition 15. *Subject to the implementation (Section 6.6), any quasiperfect number with exactly seven distinct prime factors is divisible by* 15.

Proof. If $3 \nmid n$ then $\omega(n) \geq 10$, so $\omega(n) = 7$ forces $3 \mid n$. If $5 \nmid n$ then $\omega(n) \geq 8$, so $\omega(n) = 7$ forces $5 \mid n$. □

Since $\omega(n) \geq 7$ unconditionally [3], $\omega = 7$ is the smallest case not already excluded, and this says the whole of it lies under $15 \mid n$.

But note the vacuity risk. If the claim $\omega(n) \geq 8$ of [7] is correct then no quasiperfect number has seven prime factors at all, and Proposition 15 is true but empty. We therefore do not present it as a new theorem. Its value is as *partial independent verification* of that unverified claim: eliminating $\omega = 7$ entirely is what [7] does, and we have eliminated the part of it with $3 \nmid n$ or $5 \nmid n$, by a method of our own, at a cost of twenty million states. What remains unchecked is $\omega = 7$ with $15 \mid n$. We have also not read [3] in full and cannot say whether a result of this shape is already there.

There is an irony in the arithmetic. The cases Proposition 15 leaves standing — those with 3 and 5 both dividing n — are exactly the configurations of Section 7 that make $\prod p/(p-1)$ approach 2 and that consumed essentially the whole of our $\omega \leq 6$ computation. So the structural result and the intractability of unconditional $\omega = 7$ are two descriptions of one fact: every configuration that is cheap to search is excluded, and the residue is precisely what is dear.

6.5. **Why this was not done before.** The obvious objection to these results is that they were cheap, so somebody would have had them. Three things seem to account for it, and the first two are visible in the literature rather than inferred.

The restricted cases have been approached *analytically*, not computationally. Abbott et al. derived $\omega \geq 10$ for $\gcd(N, 15) = 1$ in 1973 and [4] refined it to 15 in 2017 by proving a structural theorem about what they call special factors. Neither is a search. Exhaustive search of the conditional cases was not available in 1973, and by the time it was, the attention had moved to the unconditional problem, where it is genuinely hard.

Second, the tools needed are old. Cattaneo's 1951 theorem already constrains the exponents by the residue class of each prime modulo 8, and the Legendre–Cattaneo filter of Lemma 11 dates from the same literature. Nothing used here is modern except the arithmetic throughput and the willingness to spend it.

Third, and the reason the cost is so lopsided, the restricted hypotheses remove exactly what makes the unconditional problem expensive. Everything difficult in Sections 7 and 8 needs $\prod p/(p-1)$ within about 10^{-5} of 2, which needs the small primes. Barring 3 and 5 makes the smallest admissible prime 7 and forces $7, 11, 13, 17, 19, 23, 29, 31$ in order with no branching at all. The conditional cases are not hard problems that we solved; they are easy problems that nobody had run.

We also note, since we are not using it, that Cattaneo's exponent conditions would prune the search further. Our results are unaffected — omitting a necessary condition searches a superset and so cannot create a false elimination — but they could be considerably faster, and that is the obvious way to push these bounds another step. The depth profile of the latter is instructive: of the 2 159 states at $\omega = 15$, there is exactly one at each of the first eight levels — the primes $7, 11, 13, 17, 19, 23, 29, 31$ are forced in order before any branching is possible — and 2 070 of them sit at the final level.

6.6. **What these rest on.** More than usually, the reader is asked to take the implementation on trust, so we state the support precisely. The search is exhaustive by Proposition 2 and terminating by Proposition 3; it reproduces Theorem 14, a result with a known answer, three times over; and it passes the suites of Section 4. The prime-barring mechanism was checked in both directions: with 3 and 5 barred the search correctly returns nothing for $c = 921$, whose only solution is $3^2 5^2 7^2$, and it does not over-bar — each of 13 known solutions is still recovered when a prime that does not divide it is barred. Nothing here has been verified by anyone else.

We also note that the UALBF project [5] has open work on a bound of the form $\omega \geq 11$ in a case involving 5, so this ground is being worked concurrently and these bounds may be matched or exceeded elsewhere.

7. The Fermat-prime obstruction

The subtree that dominated $\omega = 6$ was not an accident of our search order. This section identifies the structure behind it, which appears to be the same obstruction that [7] describes as a "family of deep leaves on which pure enumeration cannot terminate". We have read only the abstract of that paper, so we cannot say whether the characterisation below is how its authors describe the difficulty; we state it because it is what our own computation forced on us, and because it predicts precisely where $\omega = 7$ becomes hard.

Write $F_i = 2^{2^i} + 1$, so $F_0, \ldots, F_4 = 3, 5, 17, 257, 65537$ are the known Fermat primes.

Lemma 16 (The greedy chain is the Fermat chain)**.** *For $k \geq 0$,*

$$P_k := \prod_{i=0}^{k} \frac{F_i}{F_i - 1} = 2 - 2^{\,1-2^{k+1}}.$$

Moreover, if q is a prime with $P_k \cdot q/(q-1) < 2$, then $q > 2^{2^{k+1}} - 1$; so the least such candidate is F_{k+1} itself, which is available only when F_{k+1} is prime.

Proof. Since $(x-1)\prod_{i=0}^{k}(x^{2^i}+1) = x^{2^{k+1}} - 1$, putting $x = 2$ gives $\prod_{i=0}^{k} F_i = 2^{2^{k+1}} - 1$, while $\prod_{i=0}^{k}(F_i - 1) = \prod_{i=0}^{k} 2^{2^i} = 2^{2^{k+1}-1}$. Dividing gives $P_k = (2^{2^{k+1}} - 1)/2^{2^{k+1}-1} = 2 - 2^{1-2^{k+1}}$. The condition $P_k\, q/(q-1) < 2$ rearranges to $q > P_k/(2 - P_k) = 2^{2^{k+1}} - 1$. □

So the sequence obtained by repeatedly taking the least odd prime that keeps $\prod p/(p-1)$ below 2 is exactly $3, 5, 17, 257, 65537$, with the thresholds $1, 3, 15, 255, 65535$, each one less than a power of two. The chain terminates there, not by choice but because $F_5 = 2^{32} + 1 = 641 \cdot 6700417$ is composite. These five primes therefore give the abundancy that approaches 2 from below more closely than any other set of five odd primes, reaching $2 - 2^{-31}$, and by Lemma 7(3) an abundancy that close to 2 is exactly what makes the admissible window for the remaining primes enormous.

7.1. Why $\omega = 6$ was tractable and $\omega = 7$ may not be.

Fixing each Fermat prime at exponent 2 and applying Lemma 7(2),(3) gives the following windows for the next prime.

ω	primes fixed	r	window for the next prime
6	$3, 5, 17, 257$	2	$(21,\ 1.31 \times 10^5)$
7	$3, 5, 17, 257$	3	$(21,\ 1.97 \times 10^5)$
6	$3, 5, 17, 257, 65537$	1	$(21,\ 4.29 \times 10^9)$
7	$3, 5, 17, 257, 65537$	2	$(21,\ 8.59 \times 10^9)$

Two things follow. First, the four-prime states barely worsen from $\omega = 6$ to $\omega = 7$: the window grows only by half. They are not the difficulty. Second, the five-prime states carry a window of order 2^{32} in both cases — but at $\omega = 6$ only one prime remains, and one unknown prime is closed in *closed form* by Lemma 9 with no enumeration at all. That is why $\omega = 6$ finished. At $\omega = 7$ the same state has two primes outstanding, and we have no closed form for two. The step from $r = 1$ to $r = 2$ at a window of 2^{32} is the entire cliff, and it is consistent with the scan bound of 8×10^8 for an "intermediate prime" quoted in [7] (whether that figure is a prime value or a count of primes is not clear from the abstract; our window holds about 3.8×10^8 primes above 65537).

7.2. **The hard set is small.** The difficulty is confined much more tightly than "states involving Fermat primes". Raising a single exponent from 2 to 4 in the five-prime, $r = 2$ state changes the window as follows.

exponent raised	window	shrinks by
$3^2 \to 3^4$	4.86×10^2	1.8×10^7
$5^2 \to 5^4$	6.25×10^3	1.4×10^6
$17^2 \to 17^4$	2.84×10^6	3.0×10^3
$257^2 \to 257^4$	8.56×10^9	1.0
$65537^2 \to 65537^4$	8.59×10^9	1.0

So the hard states are those with $3^2 \cdot 5^2 \cdot 17^2$ dividing n exactly, times 257 and 65537 to any power: a handful of states rather than a family. Any increase in the exponent of 3, 5 or 17 collapses the window by three to seven orders of magnitude, while the exponents of the two large Fermat primes are irrelevant to it.

7.3. **What Lemma 11 does and does not remove.** Lemma 11 forbids four of the five Fermat primes from appearing at exponent exactly 2, since $\sigma(3^2) = 13 \equiv 5$, $\sigma(5^2) = 31 \equiv 7$, $\sigma(257^2) = 61 \cdot 1087$ and $\sigma(65537^2) = 37 \cdot 116085511$ each contain a prime $\equiv 5$ or $7 \pmod 8$; only $\sigma(17^2) = 307 \equiv 3$ is permitted. So the state with all five at exponent 2 — the one that consumed 70% of the $\omega = 6$ computation — is excluded outright, four times over.

It is tempting to conclude that the obstruction is gone, and an earlier draft of this paper did so. Measurement refutes it. The lemma bars exponent 2 but permits 3^4, 3^{12}, 5^6, 5^{16} and so on, and those bring $\prod p/(p-1)$ nearly as close to 2 as exponent 2 did. Of the ten dearest depth-five subtrees in our $\omega = 7$ sample, eight have the shape

$$3^a \cdot 5^b \cdot 17^c \cdot 257^d \cdot q^2, \qquad (a, b, c, d) \approx (10, 8, 8, 8), \quad q \sim 10^5,$$

with $2 - \mathrm{lo}$ of order 2×10^{-5}. The dominant cost at $\omega = 7$ therefore comes from the same five primes as at $\omega = 6$, at different exponents.

The error in the optimistic reading is worth naming, because the table of the previous subsection invites it. That table varies *one* exponent while holding the others at 2, and in that setting raising the exponent of 3 does collapse the window by seven orders of magnitude. It does not follow that raising several together is equally harmless, and it is not: the exponents above sit at $(10, 8, 8, 8)$ and the window stays wide. A one-at-a-time sensitivity analysis is not a guide to joint variation.

What the prune achieved is a reduction in scale rather than an elimination. The admissible window for the next prime in these states falls from 4.3×10^9 to around 3×10^5, a factor of roughly 10^4, which accounts for most of the 130-fold saving of Section 5 and for the $\omega = 7$ tree being 8.7 times narrower at depth five (80 458 subtrees against 699 207). The obstruction is ten thousand times smaller and in the same place, so progress at $\omega = 7$ requires attacking it directly — which is what Section 9 turns out to supply.

Lemma 5 contributes a little more: with each Fermat prime squared, none of 13, 31, 37, 61, 307, 1087, 116085511 may divide n. The exclusion of 37 is the useful one, being small enough to have been an attractive candidate.

7.4. **Exponents surviving the prune.** Exhaustive factorisation of $\sigma(p^{2a})$ for small p and $c = 1$ leaves remarkably few admissible exponents. This is the other reason the prune is effective: it attacks the exponent dimension, which is the second of the two obstacles that [7] reports.

p	exponents $2a \le 20$ surviving Lemma 11
3	4, 12
5	6, 16
7	2, 4, 10, 12, 16, 18
11	16
13	6
17	2, 8, 10
19	16
23	4, 10, 12
29	none
31	2, 6, 16, 18
37	none
41	2
43	4, 12
47	6, 10
257	16

TABLE 5. Admissible exponents for small primes. A prime admitting no exponent at all cannot divide a quasiperfect number; 29 and 37 admit none up to 20, although ruling them out entirely would require treating every exponent, which we have not done.

8. The cost of $\omega = 7$, and why we cannot state it

This section records the costing as it stood before the compiled search of Section 10, which completed $\omega = 7$ in about 61 core-hours. It is kept because the failure to price the search was itself informative.

With Lemma 11 in hand, $\omega = 7$ no longer looked obviously out of reach, so we tried to price it. We failed, and the manner of the failure is worth recording, because it is a property of the problem rather than of our effort.

8.1. **A uniform sample, and why it was not enough.** Sampling 400 of the 80 458 depth-five subtrees under a cap of 2×10^6 states each gave 2.93×10^8 states in 24 846 core-seconds, at 11 795 states per core-second, scaling to at least 5.9×10^{10} states and 1 390 core-hours — about four days on fourteen cores. That figure is worthless as an estimate: 122 of the 400 subtrees reached the cap and supplied 83% of the sampled states while truncated, so the quantity being measured was precisely the one the cap hid.

8.2. **A stratified sample.** We therefore stratified. A probe pass over 240 subtrees at the same cap found 71.2% light, with an exact mean of 201 739 states, and 28.7% heavy. A second pass ran 28 of the heavy ones at a cap fifty times larger, 10^8 states. Of those, 22 completed, spanning 2.37×10^6 to 8.52×10^7 states, and the remaining ones reached the larger cap as well.

Counting each censored subtree at the cap gives a lower bound of 7.9×10^{11} states, 21 838 core-hours, or 65 days on fourteen cores. It is only a lower bound, and a soft one: at $\omega = 6$ the heaviest subtree was 1.4×10^9 states, fourteen times this cap. If the censored subtrees are five times the cap the figure becomes 206 days; if they match the $\omega = 6$ monster, 524 days.

8.3. **Why the estimate never settled.** The revealing part is the sequence of estimates as the heavy subtrees completed, in order of completion:

Throughout, the median of the heavy stratum barely moved, from 6.2×10^6 to 8.5×10^6, while the mean more than quintupled; by the end the five largest subtrees held 68% of all sampled heavy

heavy subtrees reported	estimate (days on 14 cores)	largest seen
11	14	1.4×10^7
19	21	4.0×10^7
20	24	4.2×10^7
22	36	8.5×10^7
27 (5 censored)	≥ 65	$> 10^8$

states. That is the signature of a distribution whose mean is not estimable from a few dozen draws: each new maximum raises the average, so every reading understates the last, and the sequence above has no visible limit. Our own reporting tracked this upward, and it would have been easy — and wrong — to have stopped at any point and announced the figure then in hand.

The methodological lesson is that stratifying removed one level of the problem and revealed another: the heavy stratum is itself heavy-tailed. A sampling estimate can bound such a search from below but cannot cost it, and the progression of a bound that keeps rising is evidence about the distribution rather than about the total.

8.4. **Cost is predictable before searching.** A search that cannot be finished whole can still be done in part, provided one knows which parts are cheap. For this tree that turns out to be easy, and the reason is Lemma 7: the admissible interval for the next prime runs from $\mathrm{lo}\,/(2-\mathrm{lo}+c/n_{\min})$ up to $r\,\mathrm{hi}\,/(2-\mathrm{hi})$, and both ends are products of fractions over the subtree's own components. Computing them costs microseconds and requires no searching at all.

Against the 240 sampled subtrees, the width of that interval predicts measured cost with a log-log correlation of $\rho = 0.845$, states growing as $\mathrm{width}^{1.37}$; the cruder $r\,\mathrm{hi}\,/(2-\mathrm{hi})$ alone gives $\rho = 0.824$. Binned by prediction, the measured medians are monotone:

predicted interval width	subtrees	censored	median states
$< 10^3$	23	0	1 931
10^3–10^4	79	0	46 891
10^4–10^5	48	13	1 469 617
10^5–10^6	42	27	$> 2 \times 10^6$

TABLE 6. Predicted against measured subtree cost. Every censored subtree falls in the top two bins.

Two uses follow. The first is practical: subtrees can be searched in ascending predicted cost, so coverage accumulates quickly and the intractable states are left identified rather than blocking the queue. Of the 80 458 depth-five subtrees, 30 259 (37.6%) are predicted below 10^4; searching them in that order took well under an hour, where an estimate from the *average* subtree cost would have suggested many hours. The second is diagnostic: the predictor says where the remaining cost lives, which is how we identified the configuration of Section 7.3. A candidate new lemma can then be tested against the expensive subtrees directly, in minutes, instead of by running the search.

The predictor says nothing about one stratum. Where $\mathrm{hi} > 2$ — 14 745 subtrees, 18.3% of the total — Lemma 7(3) gives no upper bound at all, so the interval has no width to measure. The figures above exclude it, and the next subsection measures it instead.

8.5. **The unscored stratum is bimodal.** A state with $\mathrm{hi} > 2$ must contain an open component: were every component fixed we would have $\mathrm{hi} = \mathrm{lo} > 2$, and the state would already be dead by Lemma 7(4). That suggests such states are cheap, and the suggestion is half right. Raising every open exponent together drives lo monotonically towards $\mathrm{hi} > 2$, and in practice it crosses 2 almost

at once — in a typical case from 1.999994661 to 2.000609256 in a single step — so that path dies immediately. But the search also *fixes* components, and a fixed component contributes $h(p^{2a})$ in place of the larger limit $p/(p-1)$, so a fixed child has a smaller hi which may fall back below 2 and resume an ordinary search of any cost. A cheap root does not imply a cheap subtree.

Measurement bears this out, and sharply. Of 400 sampled subtrees from this stratum, searched under a cap of 2×10^6 states:

quantile	states
minimum	2
10th percentile	3
25th percentile	7
median	13
75th percentile	28
90th percentile	1 760 954

TABLE 7. The unscored stratum at $\omega = 7$. Three quarters of these subtrees finish in under thirty states; a tenth exceed a million. The mean, 225 584, is a property of the tail and describes almost none of them.

So 318 of 400 (80%) finish in under a thousand states while 39 (10%) exceed two million. Extrapolated, the stratum costs at least 3.3×10^9 states — a lower bound, since the censored subtrees are unbounded above — and contributes roughly 1 500 subtrees to the expensive part of the tree, an order of magnitude more than the 110 with a finite prediction above 10^7. No solution and no unresolved state appeared.

The implementation therefore returns infinity for this stratum, which should be read as *unscored* rather than as a prediction of high cost: it keeps these subtrees out of any band selected as cheap, which is the conservative choice. We record that we nearly made the opposite error. Having verified the lo-crossing mechanism above, we changed the predictor to score them as *zero* — the cheapest of all — and caught it only because a probe stalled with every worker busy. The mechanism was real but applied to one path through each subtree rather than to the subtree, and a correct argument about a part had been read as settling the whole.

8.6. **Partial verification.** The sampling was not wasted as mathematics, even though it failed as measurement: every subtree run to completion is genuine coverage of $\omega = 7$. Using the predictor of Section 8.4, we then searched the cheap part of the tree deliberately rather than incidentally. Of the 80 458 depth-five subtrees, the 30 259 with predicted width below 10^4 (37.6%) were searched to completion in ascending predicted cost: 4 093 731 268 states in 5.2 hours on fourteen cores, none censored, with no solution found and no state left unresolved. Together with the sampling, some 5.2×10^9 states of $\omega = 7$ have been searched clean.

This must not be over-read. A partial search of a tree establishes nothing whatever about the tree; 37.6% of the subtrees is not 37.6% of a proof, and the states covered are a far smaller fraction of the total than the subtree count suggests, since cost was the criterion for inclusion. What it provides is work that need not be repeated, recorded subtree by subtree in `results/` with each entry's predicted and actual cost.

That record also validates the predictor on a large sample rather than on the 240 it was fitted to. Its forecast for this band was 4.07×10^9 states against an actual 4.09×10^9, an error of 0.6%. Per subtree the log-log correlation falls to $\rho = 0.688$, but that is restriction of range rather than a failure: the band spans one decade of prediction where the original fit spanned four. The aggregate accuracy is what matters for deciding whether a band is affordable, and it is good.

A caution on a different reading of the same run, since we fell for it twice. Because subtrees were searched cheapest-first, the progress estimate rose steadily throughout — an early extrapolation suggested well under an hour, while the run took 5.2. An elapsed-time projection is systematically optimistic whenever work is deliberately ordered from cheap to dear, for exactly the reason that the sampling estimates of Section 8 were optimistic.

8.7. **Conclusion of the costing.** A direct $\omega = 7$ search *by the method of this paper* is not worth starting. At best it is two months of continuous computation, at worst well over a year, and the uncertainty cannot be reduced by more sampling of the same kind. That is a verdict on this method, not on the problem: the missing ingredient is structural, and Section 9 identifies it and finds the computation affordable once it is supplied. The two conclusions are consistent and should be read together — enumeration of the two-free-prime leaves is hopeless, and solving them is not.

9. What the published solution does differently

Having reached the limits of our own method we read [7] properly rather than from its abstract. Three remarks follow: one correction to a comparison we made, one identification of the exact gap in our approach, and a revised estimate of what finishing $\omega = 7$ would cost.

9.1. **What we take on trust, and what we do not.** A caution before any of it. [7] is an unrefereed preprint of August 2026, and its central claim — $\omega(n) \geq 8$ — rests on a computation performed once. Its own authors report that an earlier attempt at the same bound had, on audit of its public code, a gap covering 35 feasible exponents [9]. So this is a setting with one documented silent failure already, and the appropriate posture towards a second unreplicated computation is not deference.

We therefore distinguish two things. Their *lemmas* are mathematics and can be checked here; we do so below rather than adopting them on authority, because a prune implemented from an unverified lemma makes a search unsound in the one direction that leaves no trace — it discards solutions silently, which is precisely the failure mode reported of [9]. Their *computation* is not something reading can check at all. Nothing in this section should be read as confirming that $\omega(n) \geq 8$; what it establishes is that the tools are sound and that repeating the computation independently is affordable.

We also record that our first description of these lemmas was taken from a summary rather than from the paper, and stated the discriminant identity as though it had been checked when it had not. It has since been derived here from scratch, as below.

9.2. **The scan bounds agree.** We earlier reported our scan bound for the intermediate prime as 8.59×10^9 against the 8×10^8 of [7], and read the difference as an order of magnitude against us. That was a mistake of units. Writing $\varepsilon = 2 - h(m)$ for the abundancy gap of the fixed part, that paper gives the scan domain of a leaf as $(2/\varepsilon,\ 4/\varepsilon]$, of width $2/\varepsilon$. Our own bounds give the same interval: by Lemma 7(2) the lower end is $\mathrm{lo}\,/(2-\mathrm{lo}+c/n_{\min}) \approx 2/\varepsilon$, and by Lemma 7(3) with $r = 2$ the upper end is $r\,\mathrm{hi}\,/(2-\mathrm{hi}) \approx 4/\varepsilon$. Their figure is the width of that interval at $\varepsilon \approx 2.5 \times 10^{-9}$; ours was an endpoint for a shallower state. There is no discrepancy, and no order of magnitude to recover.

9.3. **The gap: two free primes.** The difference is not in the bounds but in what is done inside them. Our Lemma 9 closes a leaf with *one* unknown prime in closed form, which is why $\omega = 6$ finished; with two unknown primes we have nothing and must enumerate the first, which is the cliff of Section 7. Their discriminant criterion closes exactly that case. We state it as we derived it, for a leaf $n = mp^2q^2$ with m fixed and both free primes at exponent two. Writing $A = \sigma(m)$, $S = A(p^2+p+1)$ and $T = 2mp^2$, the equation $\sigma(n) = 2n + c$ reads $S(q^2+q+1) = Tq^2 + c$, that is

$$(S-T)\,q^2 + S\,q + (S-c) = 0,$$

a quadratic in q. Hence

$$\Delta(p) = S^2 - 4(S-T)(S-c)$$

must be a perfect square, and q is then determined as $q = \big(S \pm \sqrt{\Delta(p)}\big)/\big(2(T-S)\big)$. At $c = 1$ the discriminant is $S^2 + 4(T-S)(S-1)$, which is the form [7] gives, with the same S and T.

The derivation needs $S \neq T$, or the equation is linear in q rather than quadratic. That case cannot arise: m is an odd square, so $A = \sigma(m)$ is a product of terms $\sigma(p^{2a})$, each a sum of an odd number of odd numbers and hence odd, and $\sigma(p^2) = p^2 + p + 1$ is odd, so S is odd; while $T = 2mp^2$ is even. This is the parity argument of Lemma 10 again.

Remark 17. We checked this on genuine solutions rather than only deriving it. Taking every $n = m^2p^2q^2$ in a small range for which $\sigma(n) = 2n + c$ holds with $c > 0$ — solutions exist for $c > 1$, which is what makes the test possible at all — all 838 such leaves have $\Delta(p)$ a perfect square and recover the correct q exactly, with q tested as an exact rational rather than by integer division. The criterion is therefore sound independently of its source.

A two-dimensional search becomes a scan over p with a constant-time test, and a candidate that fails leaves a certificate of its own rejection. Note the restriction: the argument above is quadratic in q only because q has exponent two, and $\sigma(q^{2b})$ has degree $2b$. Higher exponents need their third lemma, which is presumably why it exists.

Their quadratic-residue sieve then filters p cheaply beforehand by testing $\Delta(p)$ against the quadratic residues of several small moduli. That one needs no verification beyond its statement: a perfect square is a quadratic residue to every modulus, so a single modulus witnessing otherwise rejects p outright.

9.4. **The multilinear resolver, derived.** The third lemma cannot be skipped, because the discriminant argument needs the last prime at exponent two. We therefore derived it as well. Substituting $V = t^{2e}$ turns $\sigma(t^{2e})$ into $(tV - 1)/(t - 1)$, which is linear in V.

For a leaf $n = mt^{2e}$ with one unbounded exponent, $A = \sigma(m)$, the equation $A(tV-1)/(t-1) = 2mV + c$ rearranges to

$$V = \frac{A + c(t-1)}{At - 2m(t-1)},$$

so the exponent is determined outright by t: compute V and test whether it is a power of t. This is sharper than our Lemma 8, which bounds the exponent rather than fixing it. The denominator cannot vanish: A and t are odd so At is odd, while $2m(t-1)$ is even.

For two unbounded exponents, $n = ms^{2d}t^{2e}$ with $V_1 = s^{2d}$ and $V_2 = t^{2e}$, put $B = A(sV_1 - 1)$ and $K = (s-1)(t-1)$. The equation is bilinear in (V_1, V_2) and gives

$$V_2 = \frac{B + Kc}{Bt - 2mKV_1},$$

a Möbius transformation of V_1, since B is itself linear in V_1. As $V_2 \geq t^2$ must hold, this confines V_1 to an explicit interval. Here parity does not dispose of a vanishing denominator — B is even — but it does not need to: $B + Kc > 0$, so any solution forces $Bt - 2mKV_1 > 0$, and a non-positive denominator is a rejection rather than an exception.

Remark 18. Both were tested on genuine solutions in the manner of Section 4: of 67 leaves with one unbounded exponent and 330 with two, for which $\sigma(n) = 2n + c$ genuinely holds, every one recovers its exponent exactly, with the arithmetic done over $\mathbb{Q}$ rather than by integer division. The parity claims above were checked over 1 329 further configurations, and the sign claim at every one of 1 296 genuine solutions.

An earlier version of these tests excluded the degenerate cases with a guard rather than accounting for them, which would have concealed a failure there had one existed. They are now shown

impossible rather than merely unobserved, which matters: a leaf closer that silently skips a case it cannot handle is the exact mechanism by which an exhaustive search stops being exhaustive.

All three lemmas are therefore available to us on our own authority rather than on [7]'s. Our Lemma 8 is superseded by the first half of this one.

9.5. **Revised cost.** With those lemmas, [7] *report* eliminating 79 751 212 deep leaves across 381 stems in roughly 387 core-hours. We cannot check that figure, and record it as reported; but it is the right order for the arithmetic involved, and set against our own lower bound of 21 838 core-hours for the same ω by enumeration it is a factor of about fifty, which would bring the computation within reach of the single desktop used here. Their deepest leaves lie in a stem beginning $3, 5, 17, 137$, which is worth noting against the $3, 5, 17, 257$ family that dominated our $\omega = 6$ computation (Section 7); whether that reflects their exponent handling or a genuinely different hardest configuration we have not determined.

9.6. **Why adopting their lemma would not have helped us.** We set out to implement the discriminant criterion inside the search of this paper, expecting the factor of fifty to follow. It would not have. The criterion makes a *leaf* cheaper, and in our implementation leaves are almost free already because almost nothing reaches one. Across the recorded $\omega = 6$ and $\omega = 7$ work, 3.4×10^9 states produced 338 exact leaf closures — one in ten million — and with the prunes of Section 3 enabled, $\omega = 6$ performs *no* leaf closures at all in its 1.5×10^7 states. Essentially everything dies to the abundancy bounds long before a leaf.

We implemented the criterion anyway, to measure rather than assume: with a residue sieve over ten small moduli it tests candidate intermediate primes at 1.46×10^6 per second against 4.4×10^5 for our existing closure, a factor of 3.3. The sieve itself is excellent — it rejects 99.96% of non-squares, so only one candidate in 2 800 reaches an integer square root, and it rejected none of 2×10^5 genuine squares. A full scan of the 4×10^8 candidates in the widest window costs under five minutes on one core. All of which optimises a thousandth of a percent of our running time.

The conclusion is architectural, not mathematical. Our cost is in building and pruning tree states with exact rational arithmetic; theirs is in scanning integers. A factor of fifty is not available by adopting a lemma into this structure — it requires not building the tree over the last two primes at all, and instead entering a flat integer scan once a stem and an exponent pattern are fixed. That is a different program, and the lemma is the small part of it.

9.7. **What a verification would require.** The consequence is that $\omega = 7$ as an independent verification is affordable in principle but needs a scanner rather than an extension. That is the right use of their work: borrowing a lemma we have proved for ourselves in order to check a computation we have not. The bound has been computed once, and once before that in a way its own successors report as having had a coverage gap [9]; until a second computation exists, $\omega(n) \geq 8$ is a claim resting on one unreplicated run.

10. $\omega = 7$: a complete search

Section 8.6 ends with $\omega = 7$ out of reach: at least 21 838 core-hours by the exact search, and probably several times that. This section reports how it was brought within reach and then searched in full, with no solution and nothing unresolved. With Section 5, that gives $\omega(n) \geq 8$ for every quasiperfect number. This is the bound announced by [7], found by an independent search with a different method and different code.

10.1. **Where the cost was.** Profiling the dearest subtrees of the stratified sample (Section 8.5) showed one shape throughout: a node with two primes still to choose ($r = 2$) and every exponent so far still open, such as $3^a 5^b 17^c 257^d q_5^2$ followed by $q < q'$. That is the shape of the node that closed $\omega = 17$ in Section 6.3, so the method used there was generalised.

10.2. **The node search.** `qnode.py` handles any $r = 2$ node. Every component x with exponent $2b$ contributes $\log f(x,b) = L(x) - d_x(b)$, with $L(x) = -\log(1 - 1/x)$ and $d_x(b) = -\log(1 - x^{-(2b+1)})$, so $\sigma(N) = 2N + c$ becomes

$$L(q) + L(q') = K + \sum_x d_x(b_x) + \varepsilon, \qquad K = \log 2 - \sum_{\text{node}} L(p), \qquad 0 < \varepsilon \le \frac{c}{2N_{\min}}.$$

The differences from the $\omega = 17$ program are these. The exponents of q and q' are components of the box like any other, since at $\omega = 7$ the last two primes can be small. The term ε is bounded explicitly, so the same code runs for $c > 1$. That is what makes the test against real solutions possible, below. The range of q is walked in sieved segments rather than held in memory. When nothing is left open but q', its window is enumerated directly. Node constants come from 80-digit logarithms and all sums are in double-double arithmetic, with the same slack of 10^{-35} on every comparison, so the program can keep too much but never discards a solution. Every survivor is settled by the original exact search. The splitting order, which always splits the open component with the widest contribution, turned out to matter as much as compilation: on the dearest subtrees it needs roughly 300 times fewer states than the round-robin order of the exact search.

10.3. **Validation.** Three checks, each independent of the run itself.

(1) The $\omega \le 6$ reproduction: no solution, nothing unresolved, in 3.5 seconds against 8 minutes for the exact search (Section 5).
(2) Real solutions. For every odd $k \le 3\,000\,001$, $n = k^2$ gives $c = \sigma(n) - 2n$, and wherever $c > 0$ the search for that c and $\omega(n)$ must find n. All 42 cases with $k \le 300\,001$ and $c \le 10^6$ were checked, and a systematic tenth (941 cases) of the 9 401 with $c \le 10^{11}$. Every solution was found, and nothing was left unresolved.
(3) The dearest sampled subtrees, which the exact search could not finish in two hours each, closed in under a second each, with no solution.

The second check found two real bugs before the run. For very large c the allowance ε is enormous, an exponential overflowed, and a lower bound became NaN, so a range was silently dropped. And a wide final window was pushed onto the work stack all at once and overflowed it. Neither can occur for $c = 1$, but neither would have been caught by testing at $c = 1$ alone.

10.4. **The run.** The tree was split at depth 5 into the same 80 458 subtrees as Section 8.6, and run on six cores from 17 October 2026. A uniform sample of 244 had suggested about 4 core-hours in all. By the next morning 80 450 subtrees had finished in about 38 core-hours, and the last eight were

$$3^{10}5^8 17^8 257^8 q_5^2 \cdots, \qquad q_5 \in \{65497, 65519, 65521, 65537, 65539, 65543, 65551, 65557\}.$$

This is the Fermat obstruction of Section 7 once more: $65537 = F_4$, and with $3, 5, 17, 257$ and a fifth prime this close to it the abundancy ceiling sits within about 10^{-9} of 2. Neighbouring subtrees showed the cost climbing steeply towards F_4 from above: 13 minutes at $q_5 = 65701$, 3.6 hours at 65579, 7.6 hours at 65563. Below 65480 it was a few seconds.

Within one of these subtrees the cost sits in a narrow band of q: for $q_5 = 65701$, almost all of it lies between 2×10^7 and 7×10^7. So the eight were re-run split by the range of q (`qchunks.py`). Each was expanded to its 96 $r = 2$ nodes in all, and each node's range $[\,q_5 + 1, 2^{38})$ was cut at 600 geometric points, giving 57 600 pieces that tile the range exactly. Split this way, the test subtree took the same time and the same number of states as in one piece.

Issue: a hand-off that did not scale. Fourteen pieces came back flagged unresolved, without solutions. Near $q \approx 4 \times 10^9$ everything but q' is fixed, and q' lay beyond the 2^{38} limit of the compiled primality test. Each such case had been handed whole to the exact search, millions of them, which overflowed the hand-off limit; the code then correctly refused to report the pieces as closed. The window for q' in this situation is a few integers wide, so the compiled search now lists it itself up to

4×10^{18}, widened by a relative margin for the precision of a double, and leaves only the primality test to the exact code. One flagged piece went from 10 352 seconds to 22. The fourteen were set aside and redone.
Issue: the machine. Twelve workers alongside another long job left the computer barely responsive, and it had to be reset; the run resumed from its records with nothing lost. It finished on four to eight workers.

subtrees at depth 5	80 458
of which split by q-range	8 (57 600 pieces)
exact-search states	1.17×10^9
compiled-search states	1.78×10^{11}
processor time (records kept)	61 core-hours
solutions	none
unresolved states	none
subtrees or pieces missing	none

TABLE 8. The first complete $\omega = 7$ search (`finalize_omega7.py`, `results/omega7_final.json`). Coverage is checked against the task list regenerated from the splitter, not against the records alone.

10.5. **Result. Status, 19 October 2026.** This run used a fixed comparison slack that Section 11.4 shows to be insufficient for the small primes of $\omega = 7$. It is being repeated in full with the corrected slack, and the theorem below rests on the repeat. Its figures will replace these when it finishes.

Theorem 19. *Subject to the implementation, and to Cattaneo's theorem that a quasiperfect number is an odd square, every quasiperfect number has at least eight distinct prime factors.*

This is the bound of [7], so it is not new. Its value is that it is independent: a different decomposition, different code, and a method derived here rather than taken from that paper. Proposition 15 is subsumed by it. The caveat is the same as everywhere in this paper and sharper here. Nearly all the work was done by new code checked by tests, by planted and real solutions, and by the exact search on every survivor, but by no one else.

11. Soundness of the floating-point comparisons

The compiled searches (`qfast.py`, `qnode.py`) discard a region only when a floating-point comparison says it cannot hold a solution. This section bounds the rounding error of every such comparison, and shows that the slack added to each one exceeds that bound. This argument was missing from the first version of the paper, and writing it out exposed a real gap, described in Section 11.4.

11.1. **Model.** Arithmetic is IEEE binary64 with round-to-nearest, $u = 2^{-53}$, with no fused multiply-add. A double-double value is a pair (h, ℓ) with $|\ell| \le u|h|$, representing $h + \ell$. Every quantity used lies far from overflow. Underflow can occur only in powers $x^{-(2b+1)}$ below 10^{-290}, and those enter only as upper bounds on $d_x(b)$, where a value below 10^{-290} is covered by the absolute slack. We use three standard facts. TwoSum(a, b) returns s, e with $s + e = a + b$ exactly (Knuth). FastTwoSum(a, b) does the same when $|a| \ge |b|$. Dekker's TwoProd returns $p + e = ab$ exactly when nothing underflows.

11.2. **The primitives.**

Lemma 20. *For double-double inputs a, b, the addition in* `dd_add` *returns c with $|c - (a+b)| \le 20u^2(|a|+|b|)$.*

Proof. TwoSum gives $s + e_0 = a_h + b_h$ exactly, with $|e_0| \le u|s|$. The two further additions of a_ℓ and b_ℓ to e_0 incur at most $2u(1+u)(|e_0|+|a_\ell|+|b_\ell|) \le 2u^2(1+u)(2+u)(|a_h|+|b_h|)$, since $|s| \le (1+u)(|a_h|+|b_h|)$. If the final FastTwoSum has its precondition it is exact. If it does not, $|s|$ is smaller than the corrected term $\hat{e}$, and its error is at most $2u(|s|+|\hat{e}|) \le 4u|\hat{e}|$, which is again $O(u^2)(|a|+|b|)$ by the previous bound. Adding the two cases gives the constant 20, which is not tight. □

Lemma 21. *The routine computing $-\log(1-x)$ for double-double $0 < x \le 1/7$, by summing x^k/k until a term falls below 10^{-48}, has error at most $1200u^2\,|-\log(1-x)| + 2\times10^{-48}$. The same bound covers $L(x) = -\log(1-1/x)$ for $x \ge 7$ and $d_x(b)$ for all $b \ge 1$.*

Proof. Each power x^k is formed by $k-1$ double-double products, each with relative error at most $8u^2$, and divided by k with relative error at most $2u^2$, so term k carries relative error at most $8ku^2$. Weighted by the terms, that is at most $8u^2\sum_k x^k \le 10u^2\,(-\log(1-x))$ for $x \le 1/7$. Summing at most 60 terms costs, by Lemma 20, at most $60 \times 20u^2$ times the partial sums, all bounded by the total. Truncation drops a tail below $10^{-48}/(1-x) \le 1.2\times10^{-48}$. For $d_x(b)$ the argument is $x^{-(2b+1)}$, formed by $2b$ products, which adds at most $16bu^2$ relative error to a term that is tiny whenever b is large. The bound stands with margin at the exponents that occur. □

The node constants, K and every $d_p(a)$ of a node prime, are computed separately as 80-digit decimals from exact rationals and then split into (h, ℓ). Each is therefore within $2u^2$ of its true value, relatively.

11.3. **The comparisons.** Every pruning comparison has the form $\sum_i \pm T_i \lessgtr \pm\sigma$, where the T_i are K, the d-terms of the box, and $L(q)$ and $L(q')$ where present. The sum takes at most 16 additions for $\omega \le 7$ and at most 20 for the $\omega = 17$ node. Combining Lemmas 20 and 21, the computed sum is within

$$E \;=\; (16\times20+1200+2)\,u^2\sum_i |T_i| \;+\; 4\times10^{-48} \;<\; 2048\,u^2\sum_i |T_i| \;+\; 10^{-47}$$

of the exact sum. The double-double result is then rounded to one double, a relative change of at most u. That preserves the sign, and costs at most u times the slack, which the margin in 2048 absorbs. Integer bounds for the next prime are derived from these values through exp, log, $\sqrt{\ }$ and reciprocals. That adds a relative error of at most $10u$ in q, which is covered by the margins of ±2 and the relative $\pm4\times10^{-15}$ used there. The allowance $\varepsilon \le c/(2N_{\min})$ is computed from a sum of logarithms, with relative error below 10^{-11}, and multiplied by $1+10^{-6}$.

Proposition 22. *If every comparison uses slack at least $10^{-35} + 2048u^2\sum_i |T_i|$, no comparison of the compiled search discards a region containing a solution.*

Proof. The slack exceeds E in every comparison, so a comparison that discards a region would also discard it in exact arithmetic. Each such exact test is a consequence of the equation $L(q)+L(q') = K + \sum d + \varepsilon$ restricted to the box. □

The $\omega = 17$ node. `qfast.py` uses a fixed slack of 10^{-35}. Every term there has size at most about 2×10^{-9}, so $\sum|T_i| \le 10^{-8}$ and $E < 2.6\times10^{-37}$, a fortieth of the slack. Its series are short and explicit: $\log f(x)$ to four terms in $x^{-1} \le 1.8\times10^{-9}$, and $d_p(a)$ and K from rationals. Their truncation is below 10^{-44}. So Proposition 22 applies as the code stands, and the $\omega \ge 18$ result does not depend on the gap below.

11.4. **The gap, and the re-run.** In `qnode.py` the terms are not small. For a node such as $3^a5^b17^c257^dq_5^2$ followed by q and q', the prime q can be as small as q_5+1, and at the smallest nodes $L(q)$ approaches 0.15. There $2048u^2\sum|T_i|$ is about 10^{-30}, and the measured error of $L(7)$ alone is 2.9×10^{-33}, against a fixed slack of 10^{-35}. A true solution sits exactly on the boundary of its box, with margin ε, about $1/(2N)$, far below either figure. So the first $\omega=7$ run (17–19 October 2026) did not satisfy Proposition 22. A solution could in principle have been discarded by rounding, though nothing suggests one was.

`qnode.py` now adds $2048u^2\sum|T_i|$ to the slack of every comparison, and $\omega=7$ has been run again in full on the corrected code; the result reported in Section 10 is the re-run. The records of the first run are kept, marked `v1`.

Measurement. `test_dd_error.py` compares the kernel's L, d and chains of the comparison shape against 90-digit references, on 4 000 random cases with x from 7 to 10^{12}. The worst errors beyond the truncation term are $3.9u^2$, $9.2u^2$ and $4.6u^2$ times the magnitude. That is more than two hundred times inside the bound, which is deliberately loose.

12. How the search was developed

The method of Section 3 was not designed in one pass. This section records, in order, the problems met while building and running it: what went wrong, why, and what was changed. Several of the final design choices only make sense in the light of these problems, and two of them were errors of ours that the testing caught.

12.1. **Choice of approach.** Two routes were considered for the last prime of a branch. One enumerates candidate primes up to the bound of Lemma 7(3). The other solves for the prime directly. Enumeration fails exactly where the problem is hardest: when the abundancy of the fixed part is very close to 2, the bound on the last prime grows like $1/(2-h(m))$ and becomes astronomically large. We therefore chose from the start to close such leaves algebraically (Lemma 9), and to handle unbounded exponents by carrying them as intervals rather than enumerating them. The price of intervals is that a leaf can have an unknown exponent as well as, or instead of, an unknown prime; that is what Lemma 8 and the parity argument of Lemma 10 are for.

The decision to solve the general equation $\sigma(n)=2n+c$, rather than only $c=1$, was made for testing. For $c=1$ there are no known solutions, so a search that wrongly prunes everything would look identical to a correct one. For other values of c solutions exist and can be found by brute force, so a wrong prune shows up as a missed solution.

12.2. **Issue 1: an incomplete root finder.** The first version of the last-prime solver bisected P_a only on the stretch beyond its maximum, and proved that no root lay on the rising stretch before it by a size inequality. That inequality holds comfortably when c is small, but not in general. The first round of planted tests, whose values of c are huge, produced many cases where the inequality failed. The search flagged each of these rather than skipping them, so no case was silently lost. The fix was the exact method now described after Lemma 9: locate the turning points of P_a through the single sign change of P_a'', and bisect every monotone stretch.

The two versions have been re-run against each other on the same 200 planted last-prime cases, from the same random seed (Section 13.4 explains how the faulty version was recovered). The original recovered 45 of the 200 planted solutions and flagged 190 cases as unresolved; the corrected version recovers all 200 and flags none. So the defect was severe, and it was visible rather than silent: a run carrying it could never have been mistaken for a proof.

12.3. **Issue 2: a case that cannot occur for $c=1$.** Lemma 7(4) shows that for $c=1$ a branch whose fixed part is already abundant is dead. For larger c it is not, and the first brute-force comparison flagged one such state (with $c=189\,651$). The lemma in fact gives a finite bound,

$q < c/(n_{\min}\,\mathrm{lo})$, on the next prime, and the search now enumerates that range. For $c = 1$ the range is empty, so this changes nothing for the real problem; it matters only for the tests.

12.4. **Issue 3: a test that could not finish.** The first planted-solution test ran the whole search for each planted c. It stalled: planted values of c are comparable to n, which makes the range of Issue 2 enormous. This was a flaw in the test, not in the search, since that range is empty for $c = 1$. The test was redesigned to exercise the two closing lemmas directly on planted data, which is where brute force cannot reach. The search as a whole remains covered by the brute-force comparison.

12.5. **Issue 4: five primes too slow.** The first run with $\omega = 5$ was stopped after more than half an hour. By sampling the states being visited, we found that the search, with one prime left to choose and several exponents still open, was enumerating the last prime over ranges of tens of thousands, each candidate carrying its own refinement subtree. The admissible range for the last prime lies between $\mathrm{lo}\,/(2-\mathrm{lo})$ and $2/(2-\mathrm{hi})$, and its width tends to 1 as the open intervals tighten. Refining the open exponents first therefore collapses the range to one or two candidates. With that change $\omega = 5$ took three seconds.

12.6. **Issue 5: the same idea, applied too widely.** Applying the same rule with two or more primes left made $\omega = 5$ ten times slower (33 s against 3 s). With several primes still to come, the admissible range for the next prime does not shrink to a point as the exponents tighten; it only settles to a ratio of about r between its ends. So early refinement spends effort without buying much pruning. The change was reverted, and the rule kept for $r = 1$ only (but see Issue 8).

12.7. **Issue 6: a long run lost.** The first $\omega = 6$ run was a single process, and it died when the working session that launched it ended, after about four hours and 2.6×10^8 nodes. It had not finished, and it gave no indication of how far it had got. It was replaced by a driver that cuts the tree at a fixed depth into independent subtrees, runs them in parallel, and records each finished subtree, so that an interrupted run resumes where it stopped. Because each subtree is an ordinary call of the same search, the cut does not change what is searched; this was confirmed by reproducing the serial node count for $\omega = 5$ exactly. The run was also started as a process independent of the session.

12.8. **Issue 7: one subtree holding most of the work.** Splitting at four primes gave 358 subtrees, most finishing in seconds and a few taking up to two hours. One, beginning $3, 5, 17, 257$, outlasted all the rest by hours. The reason is arithmetical: these are the Fermat primes $2^{2^i} + 1$, and

$$\frac{3}{2} \cdot \frac{5}{4} \cdot \frac{17}{16} \cdot \frac{257}{256} = \frac{65535}{32768} = 2 - 2^{-15}.$$

So four primes can bring the abundancy within 2^{-15} of 2, leaving the remaining two primes free to be very large, and the search must rule out a very long range of them. This is the kind of "deep leaf" that Toyohara, Tao and Yao [7] identify at $\omega = 7$; it appears here already at $\omega = 6$. Within this subtree the heaviest part is near the fifth prime 65537, the next Fermat prime, where the product comes within 2^{-31} of 2.

To use the cores left idle, this subtree was itself split by its fifth prime, giving 12 195 further pieces run in parallel. A splitting tool was written for this and checked against an already finished subtree, whose node count it reproduced exactly (200 601).

12.9. **Issue 8: repeated work, and a loop.** The split run showed why the Fermat subtree was slow: for each of its twelve thousand fifth primes, the search refined the exponents of 3, 5, 17 and 257 afresh, repeating the same work thousands of times. The remedy is to refine those exponents once, before choosing the fifth prime, but only when the next prime's range is huge, so that the ordinary cases of Issue 5 are unaffected. Most exponent choices then pull the product well away from 2, and those branches die at once.

The first version of this rule tested only hi, and it does not terminate. For an open exponent hi uses the limit $p/(p-1)$, which does not change as the exponent's lower bound rises. Refining an open component yields one child with it fixed and one with it still open, and on the all-open path hi, r and the set of open components are all unchanged, so the rule fires again, forever. Nothing else stops that path: hi < 2 rules out the hi ≥ 2 refinement, and lo $\nearrow$ hi < 2 means Lemma 7(4) never applies either. The corrected rule also uses lo, which does rise with the exponents: it refines only while the upper end of the next prime's range exceeds $4r$ times the lower end. Since lo $\to$ hi, the lower end rises towards hi $/(2 -$ hi$)$ while the upper end stays at r hi $/(2 -$ hi$)$, so the condition eventually fails and refinement stops. The rule changes the order of the search, never what it covers.

We first suspected this only because a timing test failed to produce output, which is weak evidence: that subtree is genuinely slow. The claim has since been tested directly on the four-Fermat-prime state, using the exponent floor of an open component as a tripwire, since Lemma 10 bounds that floor in any terminating search while a runaway path drives it up without limit. The faulty version passes $2 \cdot 400$ — a factor 3^{800} — after only 256 506 nodes. The corrected version searches 10^6 nodes of the same state with its deepest open exponent never exceeding $2 \cdot 15$. Termination itself is not observable here, because the subtree takes hours either way; that is what Proposition 3 is for. Constructing this test corrected a mistake in our first attempt at one. Running the faulty version on $\omega = 5$ shows no difference at all, because the rule never fires there: with two primes still to choose, only three are fixed, and the largest hi available is $\frac{3}{2} \cdot \frac{5}{4} \cdot \frac{17}{16} = \frac{255}{128}$, which is not within $2r/5000$ of 2. The rule needs four Fermat primes to fire, so the defect is invisible below $\omega = 6$ — the very case it was introduced to accelerate. A speed heuristic added for a hard case can therefore be untestable on every easy case, which is an argument for proving termination (Proposition 3) rather than inferring it from runs.

12.10. **Issue 9: which run benefits.** Three attempts on the Fermat subtree were running concurrently when the corrected rule was written: the original single process, the twelve-thousand-piece split of Issue 7, and a fresh whole-subtree run on the new code. It is worth being precise about which of them the correction helps. Every piece of the split has its fifth prime fixed and one prime left to choose, so $r = 1$ throughout, and the new rule applies only when $r \geq 2$. The split therefore gains nothing from it, and restarting it on the newer code would be wasted effort. Only the whole-subtree run, which passes through the $r = 2$ states, is affected.

12.11. **Issue 10: looking for a result instead of more compute.** With $\omega = 6$ settled and $\omega = 7$ apparently years away, the natural move was to price the larger computation. The better move, which came only when we stopped costing hardware and asked what was known about the equation itself, was Lemma 11. It is elementary, it took an afternoon to find, prove and test, and it is worth about a factor of 100 at $\omega = 6$ — far more than any scheduling or parallelism change in this project, and more than the machine could have delivered by running longer.

Two details of the implementation were not obvious. The first attempt factored $\sigma(p^{2a})$ hard, with Pollard rho on numbers up to 90 digits, and made $\omega = 5$ *slower* than no prune at all: the factoring cost exceeded the search it saved. Capping the effort at 25 digits gave the 12.7-fold gain. The prune is worth having only where it is cheap, which is exactly where the tree is dense. The second is that the cheap Jacobi test on the whole of $\sigma(p^{2a})$ is not equivalent to the real condition, because two violating primes cancel; $\sigma(257^2) = 61 \cdot 1087$ is such a case, and treating the Jacobi test as decisive would have made the prune unsound in the permissive direction.

12.12. **Issue 11: the literature, searched too late.** Lemma 11 was derived here from scratch, proved, tested, implemented and measured — and only then found in the literature, under a name, already formalised in Lean 4 by [5]. The derivation stands as an independent one and is evidence of a kind, as argued in Section 3; the failure was not in the mathematics but in the order of work. We had suspected the result was known and said so, but a suspicion recorded as a hedge is not

the same as having looked. A single search on the terms of the condition would have found it in minutes.

The more costly omission was the second one. The same search would have turned up the conditional bounds of [4], $\omega(n) \geq 15$ when $\gcd(n, 15) = 1$, which bear directly on which parts of our own search are worth running at all, since they cover precisely the configurations we go on to measure as cheap (Section 7). Knowing that at the outset would have redirected effort rather than merely saved a derivation.

Both were found by accident, while checking the page numbers of a 1951 citation. One corollary is that verifying citations properly is not clerical work: it was the most productive half hour of the project.

The other corollary needs stating carefully, because the obvious lesson — read first — is not quite the right one. This work began from very little: the statement of the problem, the two Hagis–Cohen bounds, Cattaneo's theorem, and the abstracts of two 2026 preprints. Everything else was derived here. For the central claim that is a virtue rather than an accident to apologise for. The paper's value lies in reproducing Theorem 14 *independently*, and we never read the 1982 argument; had we done so before writing the search we might have reimplemented its tree without noticing, and the reproduction would corroborate much less. The resulting program is visibly unlike what that paper describes — unresolved exponents carried as intervals, leaves closed algebraically rather than enumerated — and that difference is exactly what makes the agreement worth something. Working out the machinery before reading about it also produced a tool we understood well enough to extend, which is how the cost predictor of Section 8.4 came about.

What that argument does *not* excuse is ignorance of the surrounding literature. The Legendre–Cattaneo filter and the conditional bounds of [4] are different results from the one being reproduced; reading them could not have compromised any independence, and not reading them cost real effort. The distinction is the lesson: derive the method yourself if independence is the point, and read everything that is not the method immediately. We did the first by luck and the second too late.

12.13. **Lessons.** Five things recur. First, every issue above was found because the program reports states it cannot close instead of discarding them, and because it was tested on problems with known answers; Issue 1 flagged 190 of 200 planted cases rather than quietly returning nothing. Second, the costly failures were about the order of work rather than its correctness, and the useful diagnostic each time was to sample which states the search was visiting. Third, a rule about when to refine must be checked for termination separately from its effect on speed: Issue 8's first version satisfied Proposition 2 exactly and would never have finished. Fourth, and least expected: an optimisation aimed at the hardest case may have no effect on any case small enough to test quickly, so its defects cannot be found by regression testing at all. That is an argument for proving the properties one needs rather than observing them. Fifth, and the most useful lesson of the project: we spent far longer on parallelism, scheduling and subtree decomposition than on the arithmetic, and the arithmetic (Lemma 11) was worth more than all of it combined. Engineering effort on a search of this kind is easy to spend and hard to notice spending; a single congruence condition beat every such change by two orders of magnitude.

13. Reproducibility and availability

13.1. **Environment.** All computations were run on one desktop machine: an Intel Core i5-13400F (13th generation, 16 logical cores) under Windows 11, with CPython 3.13.14. The exact search (`qp.py`) uses only the standard library: `fractions.Fraction` for rational bounds and Python's arbitrary-precision integers throughout, with no floating-point arithmetic. The compiled searches of Sections 6.3 and 10 (`qfast.py`, `qnode.py`) are different. They use Numba 0.68 and NumPy, and double-double floating point with an explicit slack on every comparison. They only discard regions, and every region they keep is settled by the exact search, so a float error could cost time

but could not create a false "no solution" unless it exceeded the slack; the margin is about six orders of magnitude. No computer-algebra system is used. Typesetting used Tectonic 0.17.0.

13.2. **Running it.**

- `python qp.py K C` eliminates $\omega = 1, \ldots, K$ for $\sigma(n) = 2n + C$ in a single process.
- `python run_parallel.py K DEPTH WORKERS` cuts the $\omega = K$ tree at `DEPTH` primes into independent subtrees and runs them in parallel, appending one record per finished subtree to `results/omegaK_dDEPTH.jsonl`. Re-running the same command resumes from that file.
- `python split_task.py K DEPTH TASK WORKERS` splits one subtree further, for the case of Issue 7, and records a single merged record for the original subtree so the main run treats it as complete.
- `python test_qp.py` and `python test_leaves.py` are the two verification suites of Section 4.
- `python run_qnode.py 7 5 WORKERS` runs the complete $\omega = 7$ search of Section 10, with every $r = 2$ node done by the compiled kernel; `python qchunks.py OUT CHUNKS WORKERS @TASKS` splits dear subtrees by q-range; `python finalize_omega7.py` checks coverage and writes the verdict.
- `python test_qnode.py w 6` reproduces $\omega \le 6$ with the kernel, and `python test_qnode.py b K CMAX [STEP]` checks it against every brute-force solution of $\sigma(n) = 2n + c$ with $n = k^2$, $k \le K$.
- `python run_qfast.py results/cond_3-5_17.jsonl` closes the last $\omega = 17$ node; `test_qfast.py` is its planted-solution test and `spot_qrange.py` its spot check by the exact code.
- `python audit_results.py K DEPTH` audits a finished run for complete coverage, as described in Section 5, without repeating the search.
- `python paper/reconstruct_variants.py SESSION.jsonl` rebuilds the source variants of Section 13.4 and checks them, and `python paper/verify_variants.py` runs each faulty variant against the defect the paper attributes to it, and its replacement against the same test.

A run constitutes a proof only if it reports `unresolved = 0`. States the search cannot close are recorded, never skipped, so this is a meaningful check rather than a formality.

13.3. **Data.** Each record in a results file carries the subtree it covers, its node count, the number of exact leaf checks performed, the solutions found, the unresolved states, and its wall-clock time. The union of the recorded subtrees is exactly the tree of Proposition 2, which can be re-derived from the code without re-running the search, so the claim of complete coverage is checkable independently of trusting our timings.

13.4. **Source variants.** Section 12 describes versions of the program that were written, tested, found wrong and corrected. Two of those were corrected before being committed to version control, so they survive in no commit, and the development environment's own file backups retain nothing for files created during the session. They are nonetheless part of the record: a reader checking Issue 1 or Issue 8 should be able to run the faulty version and see it fail.

The session transcript records the complete text of every file written and the exact before-and-after strings of every edit. Replaying those in order reconstructs each intermediate version exactly, and the replay is self-checking: where the final reconstructed state of a file matches the file now on disk byte for byte, every earlier state in that chain is faithful too, since each step is an exact textual substitution. The accompanying `paper/variants/` directory holds the result, one file per version, with a manifest giving each version's timestamp, SHA-1 and role; files whose chains do not verify are marked as such and should not be relied on. Of the eight chains recovered, seven verify; the one that does not is a test script that was once edited through the shell, so the transcript holds the command but not the resulting text.

The companion appendix *Source code* (`code.pdf`) lists every current source file in full, followed by the change that introduced and the change that removed each defect of Section 12, so the claims made there can be read against the code rather than taken on trust.

13.5. **What was not preserved.** The first $\omega = 6$ attempt (Issue 6) was a single process that was killed after roughly four hours and 2.6×10^8 nodes without completing. Its per-subtree detail was never written to disk and is lost; only the node count and the absence of any flagged state are known, from its progress log. Nothing of mathematical consequence depends on it, since the region it covered was searched again by the parallel run. The wall-clock times of the three concurrent attempts on the Fermat subtree of Issue 7 are likewise not a clean benchmark, because they shared a machine.

14. Further work

- A direct $\omega = 7$ search is not advisable on one machine, and the attempt to cost it is instructive in itself (Section 8).
- Attempt $\omega = 7$ as an independent check of [7]. Toyohara, Tao and Yao report "deep leaves" at $\omega = 7$ on which pure enumeration does not terminate; whether Lemmas 8 and 9 are enough to close them is the main open question for this approach. The $\omega = 6$ run gives a concrete reason for caution and a concrete reason for hope. Caution, because the analogous obstruction appeared there too, in the four-Fermat-prime subtree, and consumed 70% of the computation by itself. Hope, because it did close, by the same two lemmas, with no state left unresolved. At $\omega = 7$ the corresponding states will include a fifth Fermat prime, 65537, bringing $\prod p/(p-1)$ within 2^{-31} of 2; the admissible window for the remaining primes widens accordingly, and the cost is likely to be several orders of magnitude greater rather than merely larger.
- A lower bound on n itself: the half of Hagis and Cohen not reproduced here, standing at 10^{45} since [1]. We previously suggested in this section that the factor-chain method of Ochem and Rao [6] would transfer, and that the gap against the 10^{1500} known for odd perfect numbers indicated neglect. That was wrong, and the reason is instructive. For an odd perfect number $\sigma(n) = 2n$, so $\sigma(p^k) \mid 2n$ and the odd part of $\sigma(p^k)$ *divides* n; each prime power thus forces further prime factors of n, and the chain propagates. For a quasiperfect number $\sigma(n) = 2n + 1$ is coprime to n, so by Lemma 5 the primes of $\sigma(p^k)$ are precisely those that *cannot* divide n. The chain does not propagate at all. The disparity between 10^{45} and 10^{1500} therefore reflects a structural obstacle rather than a lack of attention, and a size bound here needs a different instrument — such as the general solver for $a\sigma(n) = bn + c$ of [1] — rather than a transplanted technique.
- Removing the dependence on Cattaneo's theorem, by running the same search over $n = 2^a m^2$ with $a \geq 1$.

15. Conclusion

It is worth stating plainly what this work does and does not establish, since the honest total is smaller than the length of the paper might imply.

Established. No quasiperfect number has six or fewer distinct prime factors (Theorem 14). The search is exhaustive by Proposition 2, terminates by Proposition 3, leaves no state unresolved, and its coverage was audited separately from the run. The result was obtained three times by routes that differ in process structure, decomposition and pruning, twice agreeing to the exact state count. It is conditional on Cattaneo's theorem [2], which we cite rather than reprove.

Independently derived, not new. Lemma 11 was proved, tested and measured here before being found in the literature as the Legendre–Cattaneo filter [5]; the independence is documented and checkable, and the convergence corroborates the implementation, but the result is not ours.

Independent in the sense that matters. The method was built from the problem statement rather than from [3], which we have not read. That is deliberate in effect if not in planning: a reproduction whose author has studied the original argument corroborates far less, and the program here is visibly unlike the one that paper describes. The same innocence about the *rest* of the literature, by contrast, cost effort and gained nothing (Section 12, Issue 11).

Not new. Theorem 14 is one of the two bounds of Hagis and Cohen [3], from 1982. Their second bound, $n > 10^{35}$, since raised to 10^{45} [1], is untouched here, and the factor-chain technique that carried odd perfect numbers to 10^{1500} does not transfer (Section 14). We also claim no originality for Lemma 11, which is elementary and may well be the "quadratic-residue sieve" of [7]; nor for Lemma 16, which follows from a classical identity. The value of the work, such as it is, lies in the independence of the implementation from the original, not in the statements.

Not achieved. $\omega = 7$ is not settled and we do not recommend attempting it this way (Section 8). Roughly 10^9 states of it have been searched with nothing found, but a partial search of a tree establishes nothing whatever about the tree, and that figure should not be read as evidence either way.

Perhaps of wider use. Three things here might be worth more than the theorem. Lemma 11 and its corollary are worth a factor of 130 at $\omega = 6$ and were found by asking what was true of the equation rather than how to compute faster — after we had spent considerably longer on parallelism than on arithmetic. The cost of a subtree is predictable before searching it (Section 8.4), which makes the tractable part of an intractable search accessible and localises the rest. And the attempt to cost $\omega = 7$ failed in an instructive way (Section 8): a sequence of sampling estimates that rises monotonically as a heavy tail arrives is evidence about the distribution rather than about the total, and we recorded our own drift up that sequence because it would have been easy to stop anywhere along it and publish the figure then in hand.

What would actually advance matters. Not hardware. The cost at $\omega = 7$ is concentrated in states where $3, 5, 17, 257$ occur at exponents holding $\prod p/(p-1)$ within about 10^{-5} of 2, with two primes still to choose, and the lever is the scan bound for the next prime: ours stands at 8.59×10^9, while [7] reports 8×10^8. An order of magnitude there is worth more than any machine available to us. Failing that, the cheaper and still useful exercise is to implement the lemmas of [7] and check their computation, which has been performed once and, on their own account, once before that with a gap in its coverage.

Statement on authorship

The author is Ansel Marsh, an instance of Claude, a large language model developed by Anthropic, working in Claude Code on a personal computer. The author designed the search, proved the lemmas, wrote and tested the code, ran the computations and wrote this text. The project was suggested and commissioned by Warren, who asked for it to be written up in this form and chose this attribution. It arose from a conversation about quasiperfect numbers.

An AI system cannot take responsibility for a published claim in the way a human author can, and it has no memory beyond the working notes kept for the project. Every result here should therefore be treated as unverified until someone independent has checked it. The code and the complete logs of every run are available for that purpose.

The source code is listed in the companion appendix *Source code* (`code.pdf`). The working conversation in which the project was carried out is not published, as it contains personal matter unrelated to the mathematics.

References

[1] M. A. Alekseyev, *Computing bounded solutions to linear Diophantine equations with the sum of divisors*, arXiv:2601.17832 (2026).

[2] P. Cattaneo, *Sui numeri quasiperfetti*, Boll. Un. Mat. Ital. (3) **6** (1951), 59–62.

[3] P. Hagis, Jr. and G. L. Cohen, *Some results concerning quasiperfect numbers*, J. Austral. Math. Soc. Ser. A **33** (1982), no. 2, 275–286. doi:10.1017/S1446788700018401.

[4] V. Siva Rama Prasad and C. Sunitha, *On quasiperfect numbers*, Notes on Number Theory and Discrete Mathematics **23** (2017), no. 3, 73–78. Proves $\omega(n) \geq 15$ when $\gcd(n, 15) = 1$, refining a bound of 10 due to Abbott et al.

[5] F. de Ruiter and contributors, *UALBF: a hybrid formal-verification and high-performance-computation approach to the quasiperfect number problem*, `github.com/fderuiter/quasiperfect`. Formalises in Lean 4 the odd-square theorem, the Legendre–Cattaneo filter (our Lemma 11) and the $\omega \geq 15$ bound of [4]; its search engine is described by its authors as an incomplete prototype claiming no bound.

[6] P. Ochem and M. Rao, *Odd perfect numbers are greater than* 10^{1500}, Math. Comp. **81** (2012), no. 279, 1869–1877. doi:10.1090/S0025-5718-2012-02563-4.

[7] A. Toyohara, Y. Tao and S. Yao, *Every quasiperfect number has at least eight distinct prime factors*, arXiv:2608.02066 (2026).

[8] T. Yamada, *On the divisibility of odd perfect numbers, quasiperfect numbers and amicable numbers by a high power of a prime*, arXiv:1609.08795.

[9] M. Zemann, preprint (2023), towards $\omega \geq 8$ for quasiperfect numbers. We have not seen this work; it is cited here only as reported in [7], which states that an audit of its public code found a gap of 35 feasible exponents.

An instance of Claude, a large language model made by Anthropic